



CodeEditorPlugin Diagrams

AJ McClary

July 2025

1 High-Level Architecture Diagram

1.1 Key Architectural Principles

2 Core Components Class Diagram

2.1 Key Design Patterns

3 Configuration System Diagram

3.1 Configuration Usage Patterns

4 Service Architecture Diagram

4.1 Service Architecture Principles

5 Event System Flow Diagram

5.1 Event System Code Examples

5.2 Key Features

6 Language Support & Syntax Highlighting Pipeline

6.1 Language Configuration Example

6.2 Performance Optimizations

7 Completion System Architecture

7.1 Completion Flow

7.2 Key Features

8 Platform Abstraction Layer

8.1 Platform-Specific Code Example

8.2 Key Abstraction Principles

9 Text Processing Pipeline

- 9.1 Detailed Processing Steps
- 9.2 Performance Characteristics
- 9.3 Key Optimizations
- 10 UI Component Hierarchy
 - 10.1 Layout Structure
 - 10.2 Component Responsibilities
- 11 Advanced Features Integration Architecture
 - 11.1 Key Features Integration
 - 11.2 Architecture Benefits
- 12 LSP System Complete Architecture
 - 12.1 LSP System Flow
 - 12.2 Key LSP Features
 - 12.3 Benefits
- 13 Performance Monitoring & Optimization System
 - 13.1 Performance Optimization Flow
 - 13.2 Key Performance Features
 - 13.3 Benefits
- 14 Symbol Navigation & Code Intelligence
 - 14.1 Symbol Navigation Flow
 - 14.2 Key Symbol Intelligence Features
 - 14.3 Benefits
- 15 Language Provider Complete Ecosystem
 - 15.1 Language Support Matrix
 - 15.2 Key Ecosystem Features
 - 15.3 Benefits
- 16 Annotation System Detailed Architecture
 - 16.1 Annotation System Flow

16.2 Key Annotation Features

16.3 Benefits

17 Advanced Text Processing & Validation Pipeline

17.1 Text Processing Flow

17.2 Key Text Processing Features

17.3 Benefits

18 Data Models & Type System Architecture

18.1 Data Model Interaction Flow

18.2 Key Data Model Features

18.3 Benefits

19 SwiftUI Integration Complete Ecosystem

19.1 SwiftUI Integration Flow

19.2 Key SwiftUI Integration Features

19.3 Usage Examples

19.4 Benefits

20 Debugging Integration Detailed Architecture

20.1 Debug Integration Flow

20.2 Key Debugging Features

20.3 Benefits

21 Utility Systems & Extensions Network

21.1 Utility System Integration Flow

21.2 Key Utility System Features

21.3 Usage Examples

21.4 Benefits

22 Advanced Layout & UI Components Architecture

22.1 Layout System Flow

22.2 Key Layout System Features

22.3 Editor-Specific Components

22.4 Benefits

23 Multi-Language Support Matrix

23.1 Language Capability Details

23.2 Language Support Tiers

23.3 Feature Comparison Matrix

23.4 Performance Characteristics

23.5 Extension Strategy

23.6 Benefits

24 Performance Optimization Pipeline

24.1 Detailed Performance Architecture

24.2 Performance Optimization Flow

24.3 Key Performance Features

24.4 Optimization Strategies

24.5 Benefits

25 Color Palette - Universal Light/Dark Mode

25.1 Core System Colors (Light/Dark Mode Compatible)

25.2 Accent Colors (Adaptive)

25.3 Neutral Grays (Adaptive)

25.4 Semantic Colors (Context-Aware)

26 CodeEditorPlugin Architecture Diagrams

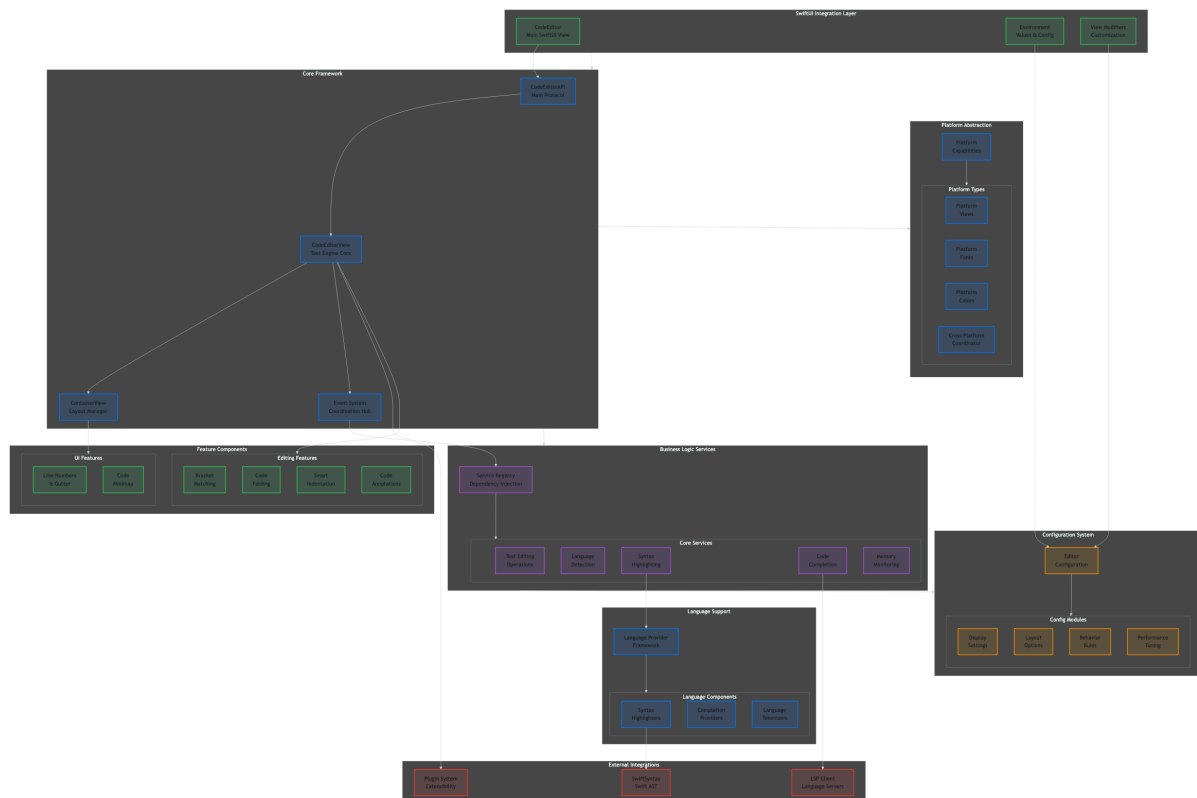
26.1 Index of Diagrams

26.2 How to View These Diagrams

26.3 Diagram Conventions

1 High-Level Architecture Diagram

This diagram shows the overall architecture of the CodeEditorPlugin framework, illustrating the main layers and their relationships.



1.1 Key Architectural Principles

1. **Layered Architecture:** Clear separation between UI (SwiftUI), Core logic, Services, and Platform abstractions
2. **Protocol-Oriented:** Core functionality defined through protocols (CodeEditorAPI)
3. **Service-Based:** Business logic encapsulated in services managed by a central registry
4. **Platform Agnostic:** Platform-specific code isolated in abstraction layer
5. **Event-Driven:** Unified event system for decoupled communication
6. **Configurable:** Comprehensive configuration system with environment

integration

7. **Extensible:** Plugin system and language provider architecture for extensions

2 Core Components Class Diagram

This diagram details the main classes and protocols that form the core of the CodeEditorPlugin framework.

2 Core Components Class Diagram

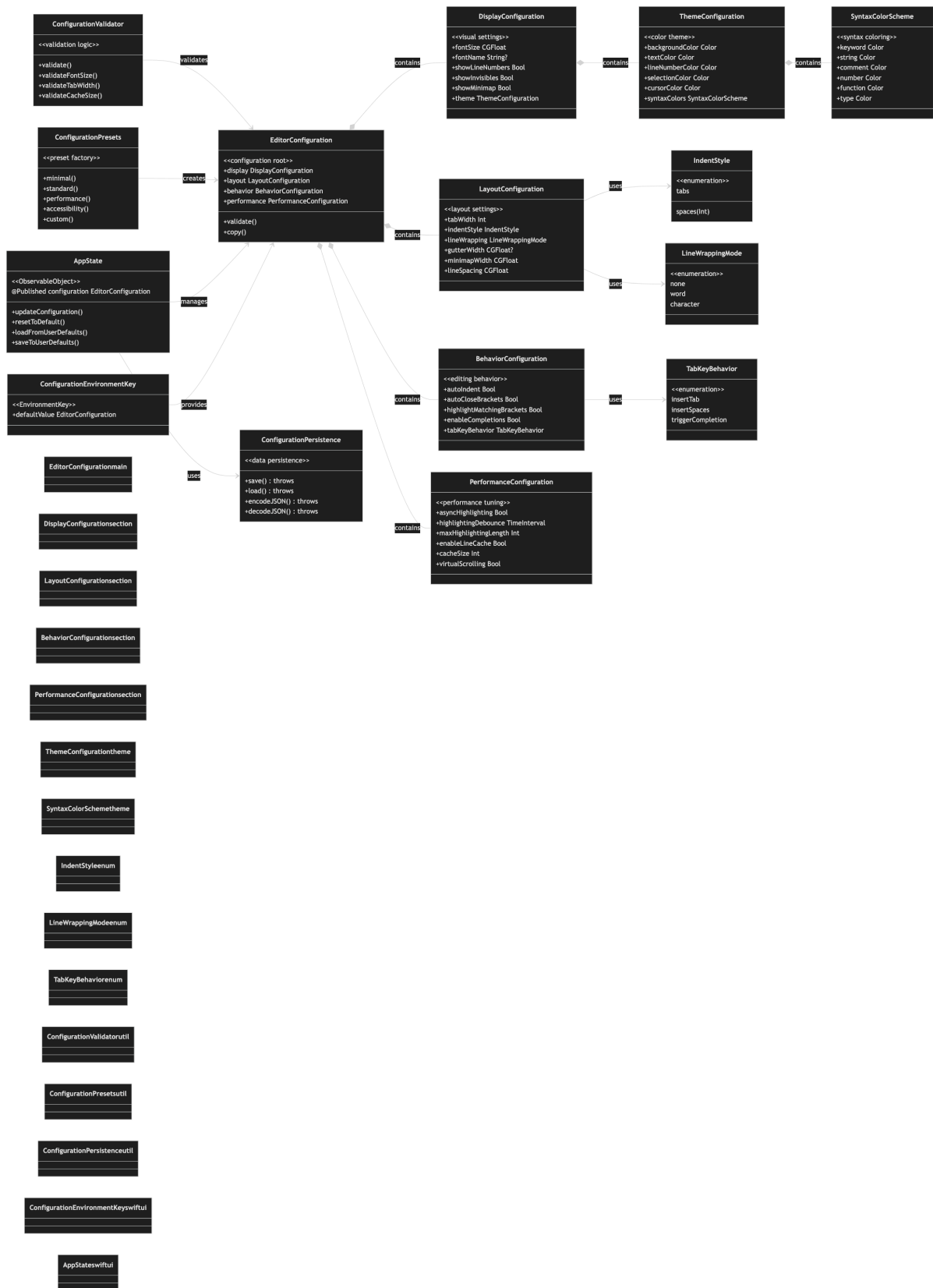


2.1 Key Design Patterns

1. **Protocol-Oriented Design:** Core functionality exposed through `CodeEditorAPI` protocol
2. **Delegation Pattern:** `CodeEditorViewDelegate` for customizable behavior
3. **Service Locator:** `BusinessLogicServiceRegistry` manages all services
4. **Observer Pattern:** `UnifiedEventSystem` for decoupled event handling
5. **Composite Pattern:** `CodeEditorContainerView` composes multiple views
6. **Bridge Pattern:** `CodeEditor` SwiftUI wrapper bridges to `AppKit/UIKit`
7. **Cache Pattern:** `LineIndexCache` for performance optimization

3 Configuration System Diagram

This diagram illustrates the comprehensive configuration system used throughout the CodeEditorPlugin framework.



3.1 Configuration Usage Patterns

3.1.1 Direct Property Access

```
config.display.showLineNumbers = true  
config.layout.tabWidth = 4
```

3.1.2 Batch Updates

```
appState.updateConfiguration { config in  
    config.display.fontSize = 16  
    config.behavior.autoIndent = true  
}
```

3.1.3 Environment Integration

```
CodeEditor(text: $code)  
    .environment(\.codeEditorConfiguration, customConfig)
```

3.1.4 Presets

```
let minimalConfig = EditorConfiguration.minimal  
let performanceConfig = EditorConfiguration.performance
```

4 Service Architecture Diagram

This diagram shows the service-oriented architecture and how services interact within the CodeEditorPlugin framework.



4.1 Service Architecture Principles

1. **Service Registry Pattern:** Central registry manages all service instances
2. **Dependency Injection:** Services receive dependencies through constructor
3. **Lifecycle Management:** All services implement lifecycle protocol
4. **Event-Driven Communication:** Services communicate through event system
5. **Caching Strategy:** Shared cache manager for performance
6. **Async Operations:** Heavy operations run on background queues
7. **Memory Management:** Automatic cleanup on memory warnings

5 Event System Flow Diagram

This diagram illustrates the unified event system that enables decoupled communication between components.

5.1 Event System Code Examples

5.1.1 Event Definition

```
struct Event {  
    let type: EventType  
    let source: Any  
    let timestamp: Date  
    let data: [String: Any]  
}  
  
enum EventType {  
    case textChanged  
    case selectionChanged  
    case languageChanged  
    case configurationChanged  
    case memoryWarning  
    case completionRequested  
    case highlightingCompleted  
}
```

5.1.2 Handler Registration

```
eventSystem.register(.textChanged, priority: .high) { event in  
    // Handle text change  
    guard let range = event.data["range"] as? NSRange else {  
        return }  
    // Process change...  
}
```

5.1.3 Event Emission

```
eventSystem.emit(Event(  
  type: .textChanged,  
  source: self,  
  timestamp: Date(),  
  data: ["range": range, "text": newText]  
))
```

5.1.4 Event Filtering

```
eventSystem.addFilter { event in  
  // Only process events from specific sources  
  return event.source is CodeEditorView  
}
```

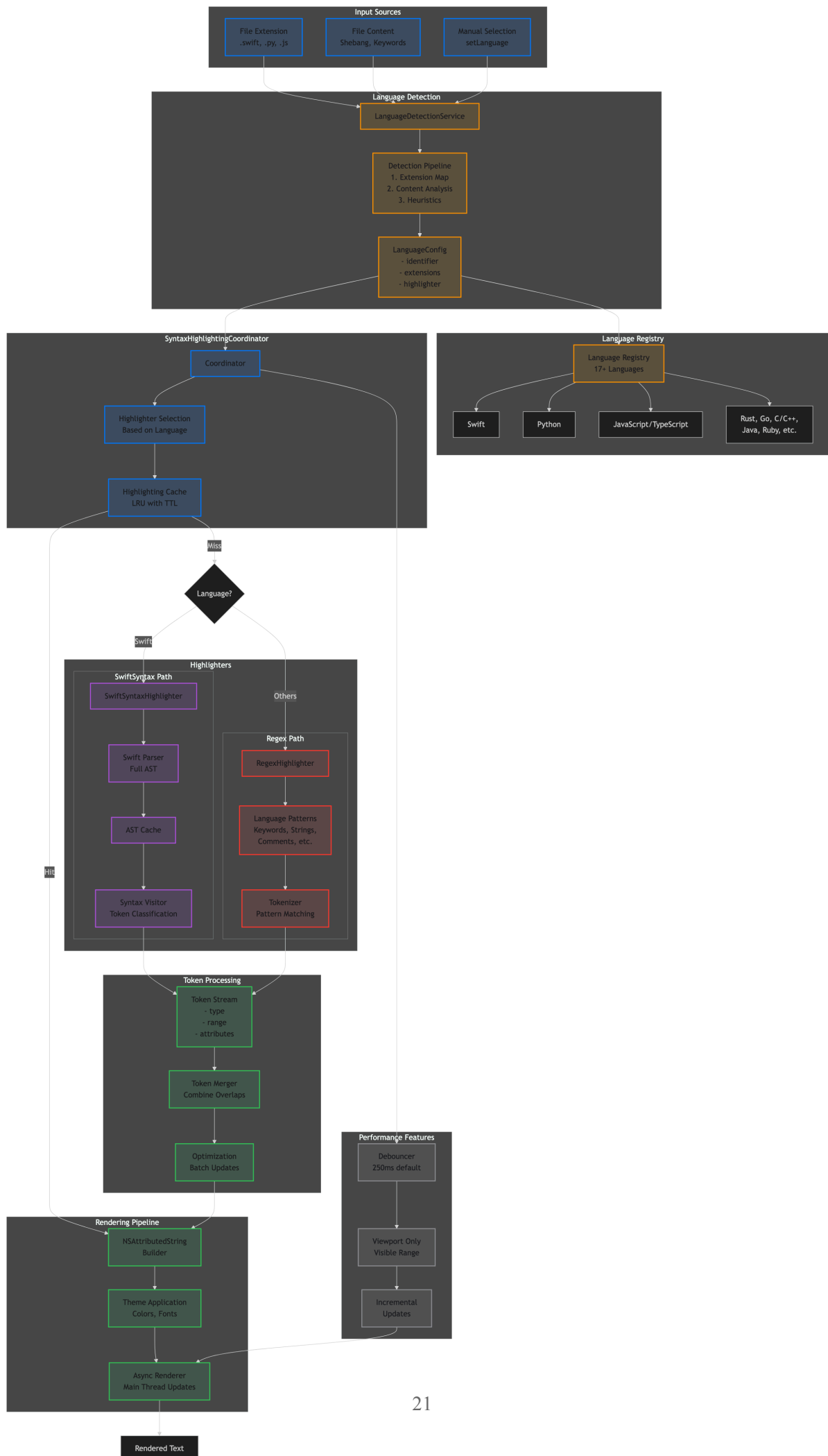
5.2 Key Features

1. **Priority-based Processing:** High-priority events processed first
2. **Asynchronous Support:** Long-running handlers don't block UI
3. **Event Filtering:** Reduce unnecessary processing
4. **Batch Processing:** Efficient handling of multiple related events
5. **Error Isolation:** Handler errors don't crash the system
6. **Event Chaining:** Handlers can emit new events
7. **Performance Monitoring:** Track handler execution times

6 Language Support & Syntax Highlighting Pipeline

This diagram shows the complete pipeline for language detection and syntax highlighting, including both SwiftSyntax and regex-based paths.

6 Language Support & Syntax Highlighting Pipeline



6.1 Language Configuration Example

```
struct LanguageConfig {  
    let identifier: String  
    let displayName: String  
    let fileExtensions: [String]  
    let highlighterType: HighlighterType  
    let completionProvider: CompletionProvider?  
    let indentationRules: IndentationRules  
}  
  
enum HighlighterType {  
    case swiftSyntax  
    case regex(patterns: LanguagePatterns)  
}
```

6.2 Performance Optimizations

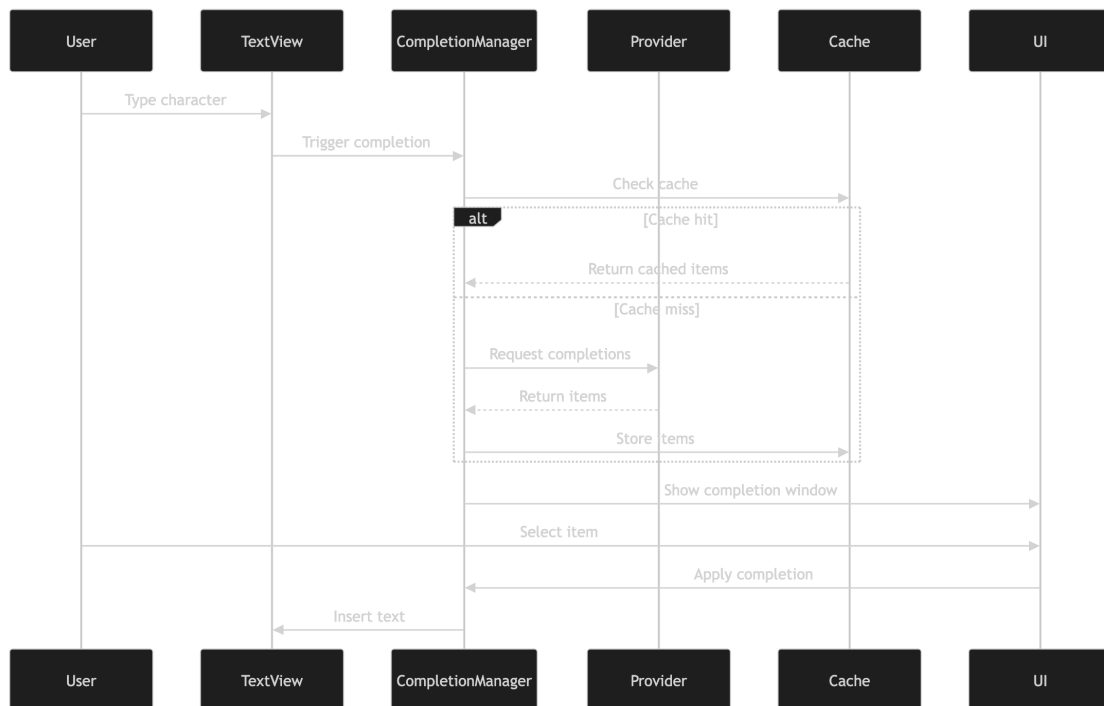
1. **AST Caching:** SwiftSyntax ASTs cached for reuse
2. **LRU Cache:** Recently highlighted documents cached
3. **Viewport Rendering:** Only visible text highlighted
4. **Incremental Updates:** Only changed regions re-highlighted
5. **Debouncing:** Rapid changes batched together
6. **Background Processing:** Heavy parsing off main thread
7. **Token Batching:** Multiple tokens applied in single update

7 Completion System Architecture

This diagram shows the code completion system architecture, including provider management, caching, and UI integration.



7.1 Completion Flow



7.2 Key Features

1. **Multi-Provider Support:** Different providers for different languages
2. **Async Processing:** Non-blocking completion requests
3. **Smart Caching:** LRU cache with context-aware invalidation
4. **Debouncing:** Prevents excessive completion requests
5. **Filtering & Sorting:** Client-side filtering of results
6. **LSP Integration:** Full Language Server Protocol support
7. **Snippet Expansion:** Template-based code snippets

8 Platform Abstraction Layer

This diagram shows how the CodeEditorPlugin achieves cross-platform compatibility across macOS, iOS, and Mac Catalyst.



8.1 Platform-Specific Code Example

```
// Platform detection
#if canImport(AppKit)
import AppKit
typealias PlatformView = NSView
typealias PlatformColor = NSColor
#elseif canImport(Uikit)
import UIKit
typealias PlatformView = UIView
typealias PlatformColor = UIColor
#endif

// Feature detection
if PlatformCapabilities.shared.hasHoverSupport {
    // Enable hover highlighting
}

// Platform-specific behavior
switch CrossPlatformCoordinator.shared.currentPlatform {
case .macOS:
    // macOS-specific implementation
case .iOS:
    // iOS-specific implementation
case .macCatalyst:
    // Catalyst-specific implementation
}
```

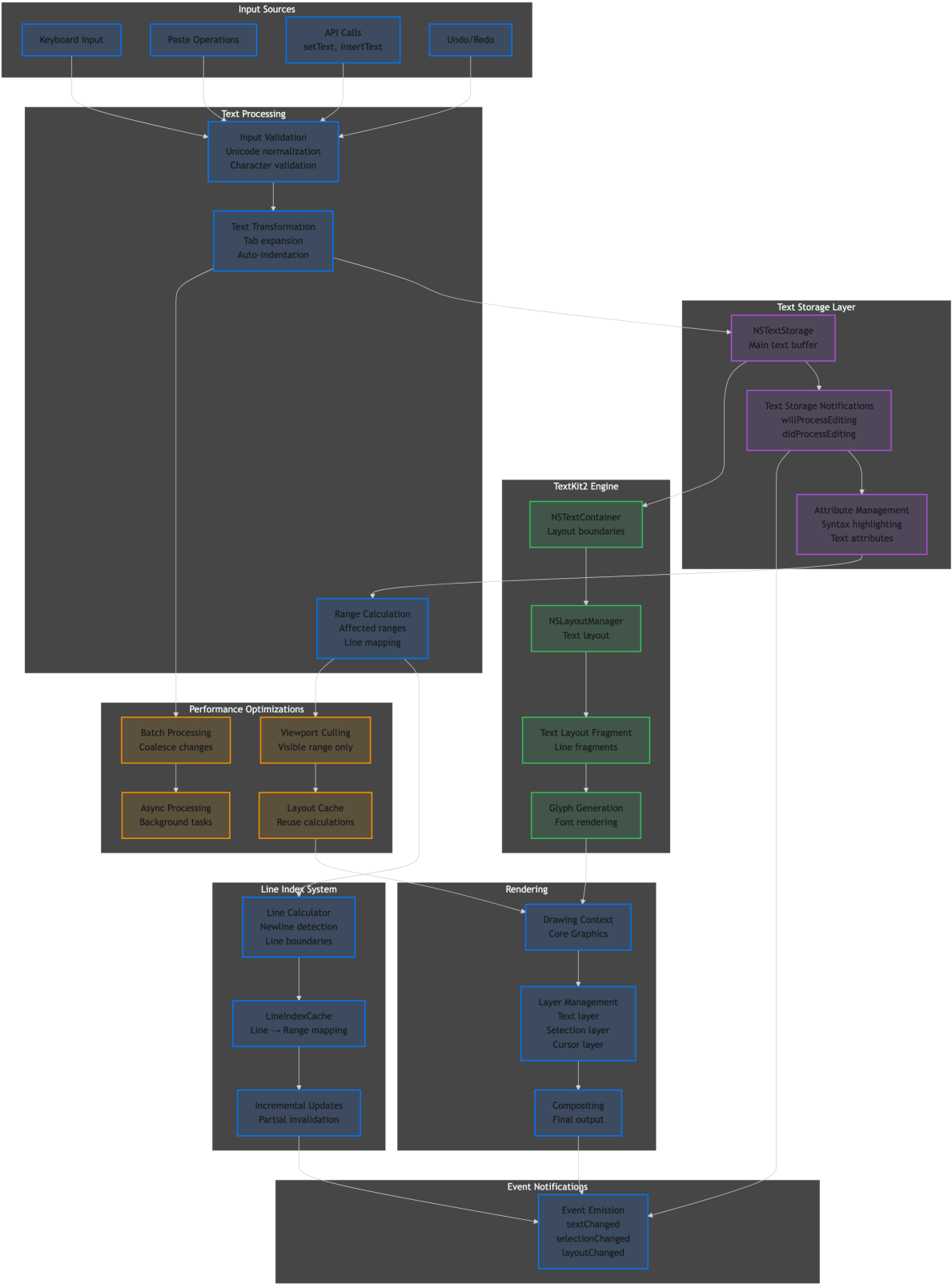
8.2 Key Abstraction Principles

1. **Compile-time Type Aliases:** Use `typealias` for platform types
2. **Runtime Feature Detection:** Check capabilities at runtime
3. **Protocol-based Abstractions:** Define common interfaces
4. **Adapter Pattern:** Convert platform-specific events to unified format

5. **Factory Pattern:** Create platform-specific implementations
6. **Conditional Compilation:** Use `#if canImport()` for platform code
7. **Graceful Degradation:** Disable unsupported features cleanly

9 Text Processing Pipeline

This diagram illustrates the text processing pipeline, from input to rendering, including TextKit2 integration and performance optimizations.



9.1 Detailed Processing Steps

9.1.1. Input Validation

```
func validateInput(_ text: String) -> String {  
    // Unicode normalization  
    let normalized = text.precomposedStringWithCanonicalMapping  
  
    // Remove control characters  
    let cleaned = normalized.filter { !$0.isControl }  
  
    return cleaned  
}
```

9.1.2. Line Index Management

```
class LineIndexCache {  
    private var lineStarts: [Int] = []  
    private var version: Int = 0  
  
    func updateForTextChange(range: NSRange, delta: Int) {  
        // Incremental update algorithm  
        let startLine = lineContaining(location: range.location)  
        // Update only affected lines...  
    }  
}
```

9.1.3 3. Batch Processing

```

class BatchProcessor {
    private var pendingChanges: [TextChange] = []
    private var timer: Timer?

    func addChange(_ change: TextChange) {
        pendingChanges.append(change)
        scheduleBatchProcessing()
    }

    private func processBatch() {
        textStorage.beginEditing()
        pendingChanges.forEach { apply($0) }
        textStorage.endEditing()
    }
}

```

9.2 Performance Characteristics

Operation	Complexity	Optimization
Text Insertion	$O(n)$	Batch updates
Line Lookup	$O(1)$	Cached indices
Syntax Highlight	$O(n)$	Viewport only
Layout	$O(\text{visible})$	Virtualization
Scrolling	$O(1)$	Pre-calculated

9.3 Key Optimizations

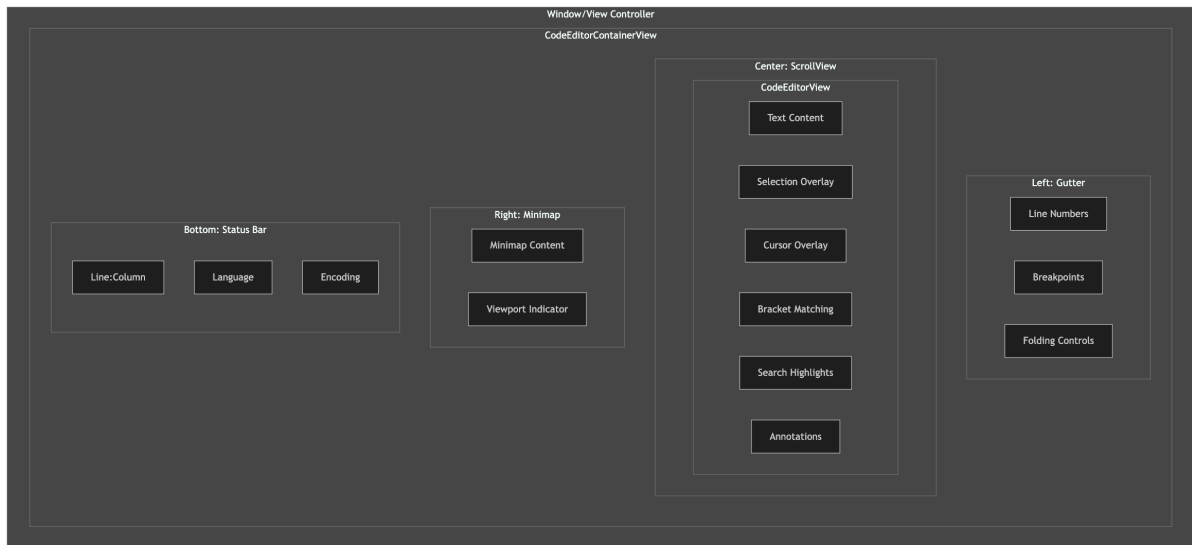
1. **Incremental Updates:** Only process changed regions
2. **Line Caching:** $O(1)$ line number lookups
3. **Viewport Rendering:** Only layout visible text
4. **Batch Processing:** Coalesce rapid changes
5. **Async Highlighting:** Non-blocking syntax updates
6. **Glyph Caching:** Reuse font measurements
7. **Layer Composition:** Separate layers for different elements

10 UI Component Hierarchy

This diagram shows the visual component hierarchy and layout structure of the CodeEditorPlugin.



10.1 Layout Structure



10.2 Component Responsibilities

1. **Container View:** Manages overall layout and coordinates subviews
2. **Gutter View:** Displays line numbers, breakpoints, and folding controls
3. **Editor View:** Main text editing area with TextKit2 integration
4. **Minimap View:** Provides document overview and navigation
5. **Overlay Views:** Handle selections, cursor, and visual feedback
6. **Scroll View:** Manages scrolling and viewport synchronization
7. **Status Bar:** Shows current editor state and file information

11 Advanced Features Integration Architecture

This diagram shows the advanced features system that extends the core CodeEditorPlugin functionality with debugging, search/replace, smart editing, code folding, and symbol navigation capabilities.



11.1 Key Features Integration

11.1.1 1. Debugger Integration

- **Full DAP Support:** Debug Adapter Protocol implementation
- **Breakpoint Management:** Visual breakpoints with validation
- **Variable Inspection:** Expression evaluation and variable watching
- **Execution Control:** Step-by-step debugging with full control

11.1.2 2. Search & Replace Engine

- **Multi-mode Search:** Text, regex, symbol, and file search
- **Advanced Replace:** Single and batch replace with undo/redo
- **Search History:** Previous searches with quick access
- **Scope Control:** Document, selection, or project-wide search

11.1.3 3. Smart Editing Engine

- **Context-aware Completion:** Enhanced completion with priority calculation
- **Intelligent Indentation:** Language-specific smart indentation
- **Auto-completion:** Bracket matching and electric character handling
- **Code Actions:** Quick fixes and refactoring suggestions

11.1.4 4. Code Folding System

- **Multi-provider Support:** Brace, indentation, and comment folding
- **Language Agnostic:** Works with any supported language
- **Visual Integration:** Seamless UI integration with gutter
- **Persistent State:** Folding state preserved across sessions

11.1.5 5. Symbol Navigation

- **Document Outline:** Hierarchical symbol view
- **Breadcrumb Navigation:** Current scope breadcrumbs

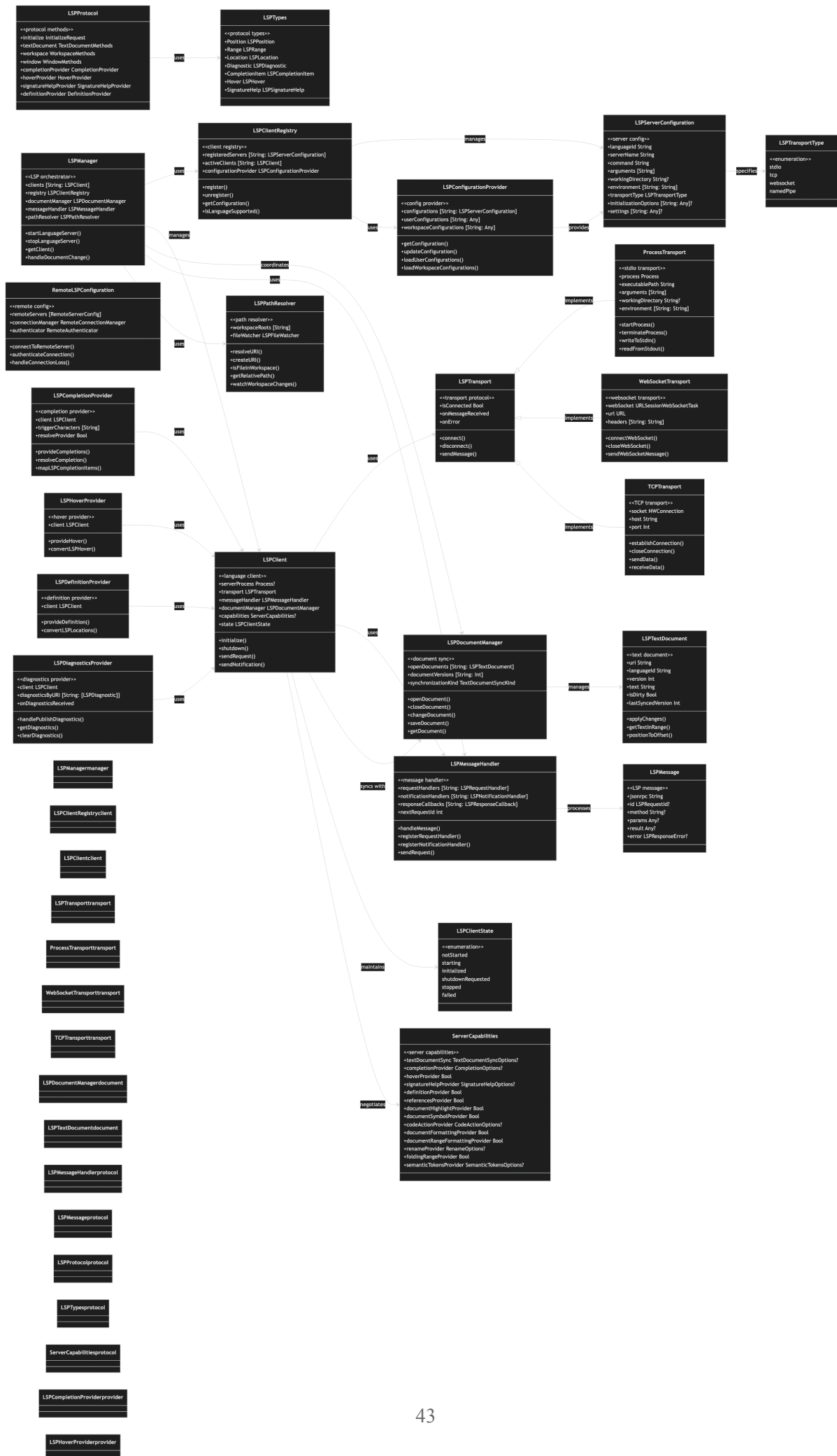
- **Go-to Definition:** Quick symbol navigation
- **Find References:** Symbol usage across codebase

11.2 Architecture Benefits

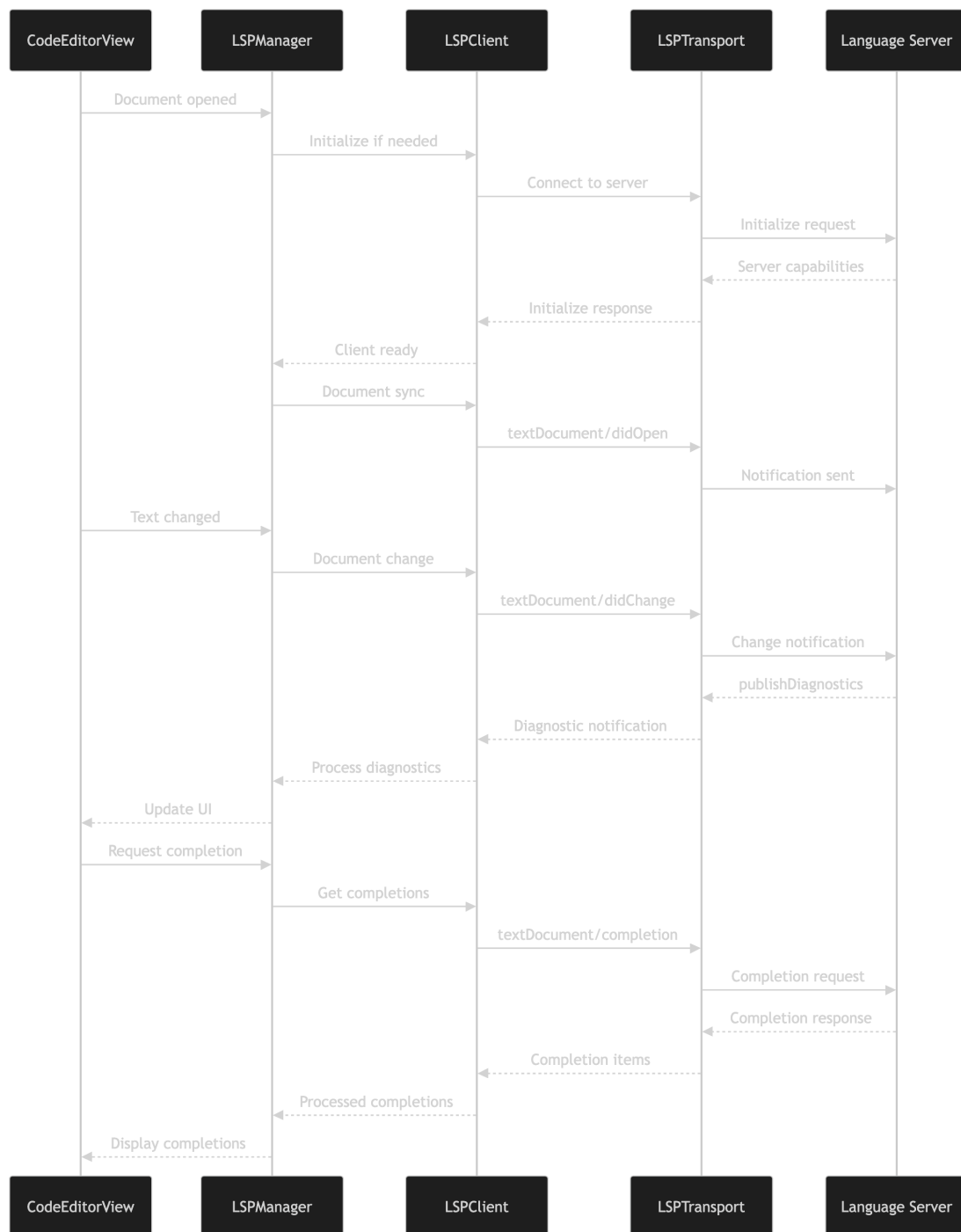
1. **Modular Design:** Each feature can be enabled/disabled independently
2. **Language Support:** Features work across all supported languages
3. **Performance Optimized:** Lazy loading and efficient caching
4. **Extensible:** Plugin architecture for additional features
5. **Integration Ready:** Seamless integration with LSP and other systems

12 LSP System Complete Architecture

This diagram shows the comprehensive Language Server Protocol implementation that provides advanced language features through external language servers.



12.1 LSP System Flow



12.2 Key LSP Features

12.2.1 1. Multi-Transport Support

- **Process Transport:** Standard stdio-based communication
- **WebSocket Transport:** Web-based language servers
- **TCP Transport:** Network-based language servers
- **Named Pipe Transport:** IPC-based communication

12.2.2 2. Document Synchronization

- **Full Sync:** Complete document content synchronization
- **Incremental Sync:** Delta-based change synchronization
- **Version Management:** Document version tracking
- **Change Batching:** Efficient change aggregation

12.2.3 3. Feature Providers

- **Completion Provider:** LSP-based code completion
- **Hover Provider:** Symbol information on hover
- **Definition Provider:** Go-to-definition functionality
- **Diagnostics Provider:** Error and warning reporting
- **Formatting Provider:** Code formatting support

12.2.4 4. Configuration Management

- **Server Registry:** Automatic server discovery
- **User Configuration:** Customizable server settings
- **Workspace Configuration:** Project-specific settings
- **Remote Support:** Cloud-based language servers

12.2.5 5. Reliability Features

- **Connection Recovery:** Automatic reconnection on failure

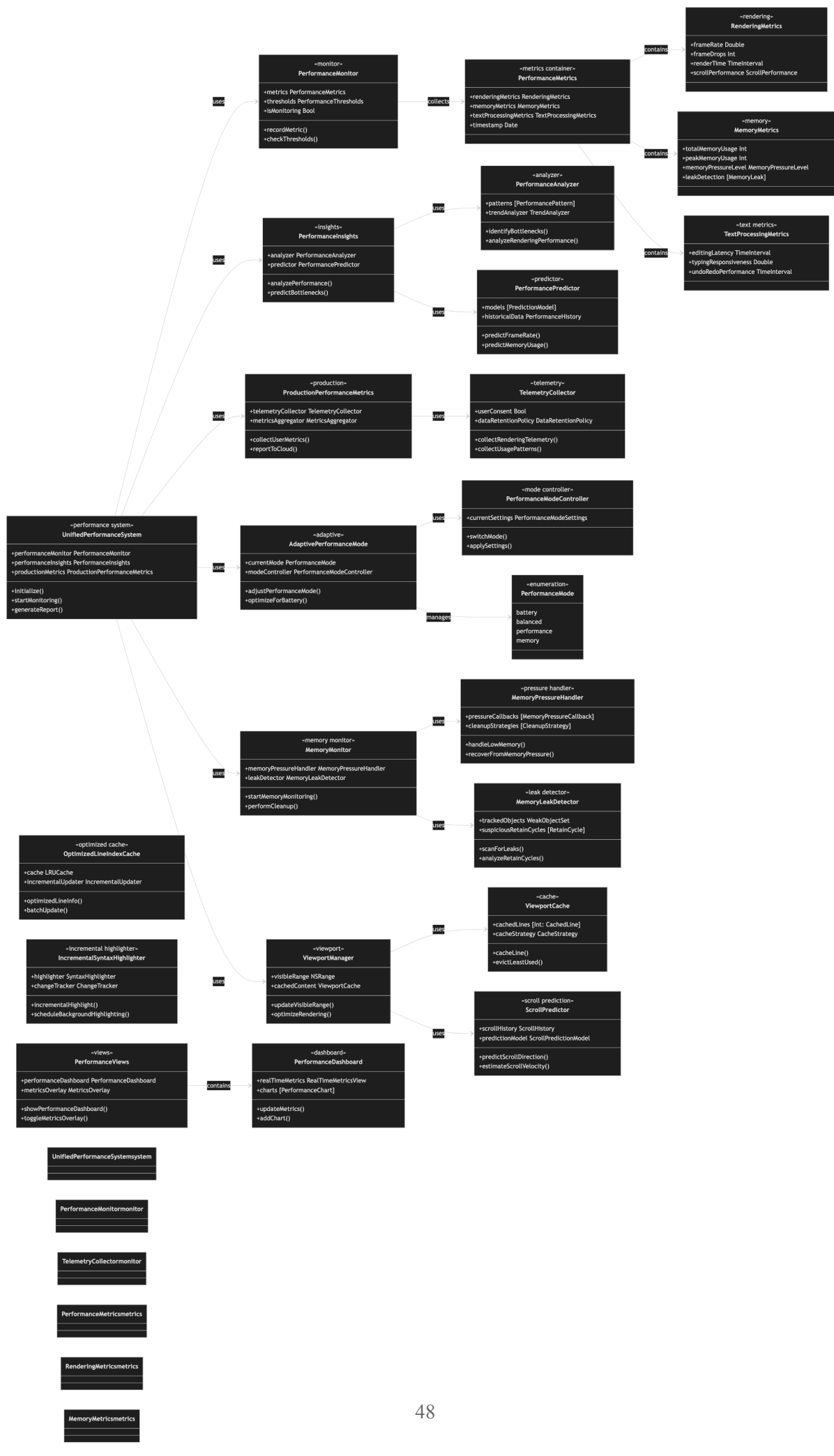
- **Error Handling:** Comprehensive error management
- **State Management:** Robust client state tracking
- **Resource Cleanup:** Proper resource disposal

12.3 Benefits

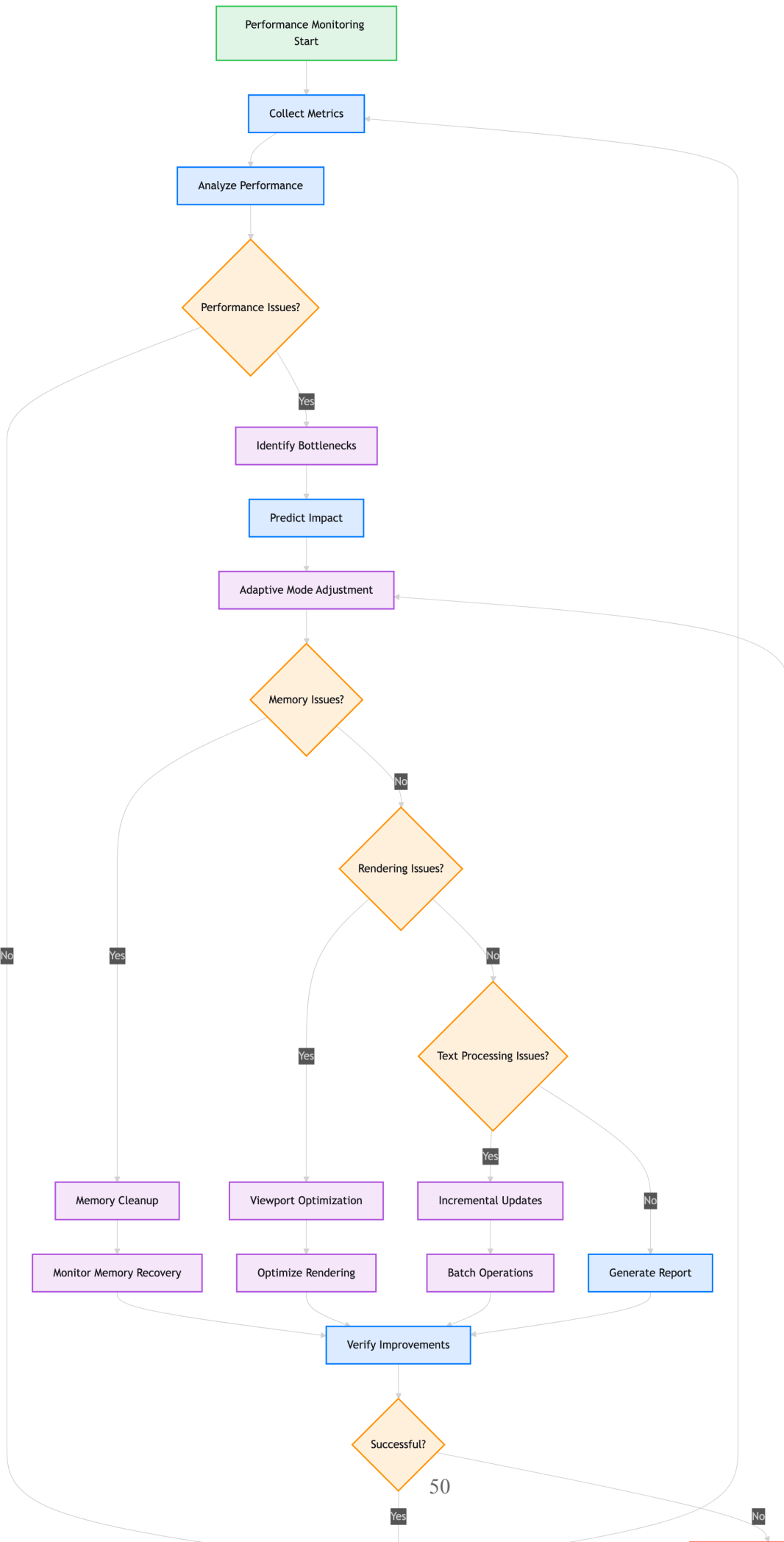
1. **Language Agnostic:** Works with any LSP-compliant language server
2. **Scalable:** Handles multiple concurrent language servers
3. **Robust:** Built-in error handling and recovery
4. **Extensible:** Plugin architecture for custom providers
5. **Performance:** Optimized message handling and caching

13 Performance Monitoring & Optimization System

This diagram shows the comprehensive performance monitoring and optimization system that ensures 60fps rendering and efficient resource usage.



13.1 Performance Optimization Flow



13.2 Key Performance Features

13.2.1 1. Real-time Monitoring

- **Frame Rate Tracking:** Continuous 60fps monitoring
- **Memory Usage:** Real-time memory pressure detection
- **Processing Times:** Edit latency and responsiveness tracking
- **Resource Utilization:** CPU and memory usage optimization

13.2.2 2. Adaptive Performance

- **Dynamic Mode Switching:** Automatic performance mode adjustment
- **Resource-based Optimization:** Battery/performance/memory optimized modes
- **Contextual Adaptation:** File size and complexity based adjustments
- **User Preference Integration:** Customizable performance profiles

13.2.3 3. Memory Management

- **Pressure Handling:** Automatic cleanup on memory warnings
- **Leak Detection:** Proactive memory leak identification
- **Smart Caching:** Efficient viewport and content caching
- **Garbage Collection:** Optimized memory reclamation

13.2.4 4. Viewport Optimization

- **Visible Range Management:** Only render visible content
- **Predictive Loading:** Preload content based on scroll patterns
- **Efficient Invalidation:** Minimal redraw operations
- **Smart Caching:** Cache frequently accessed content

13.2.5 5. Performance Analytics

- **Bottleneck Identification:** Automatic performance issue detection

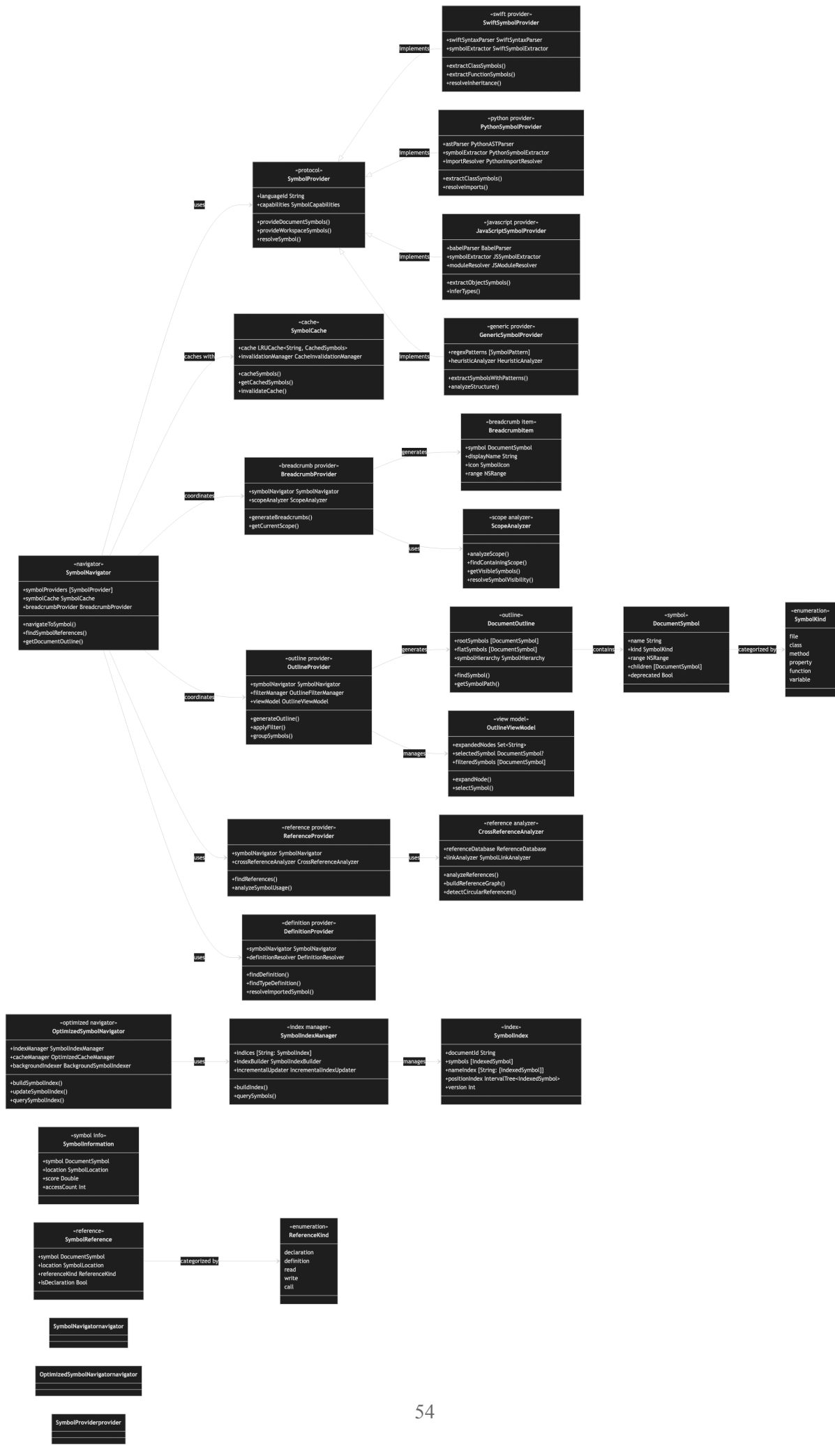
- **Trend Analysis:** Long-term performance pattern analysis
- **Predictive Analytics:** Performance prediction based on context
- **Optimization Suggestions:** Automated performance recommendations

13.3 Benefits

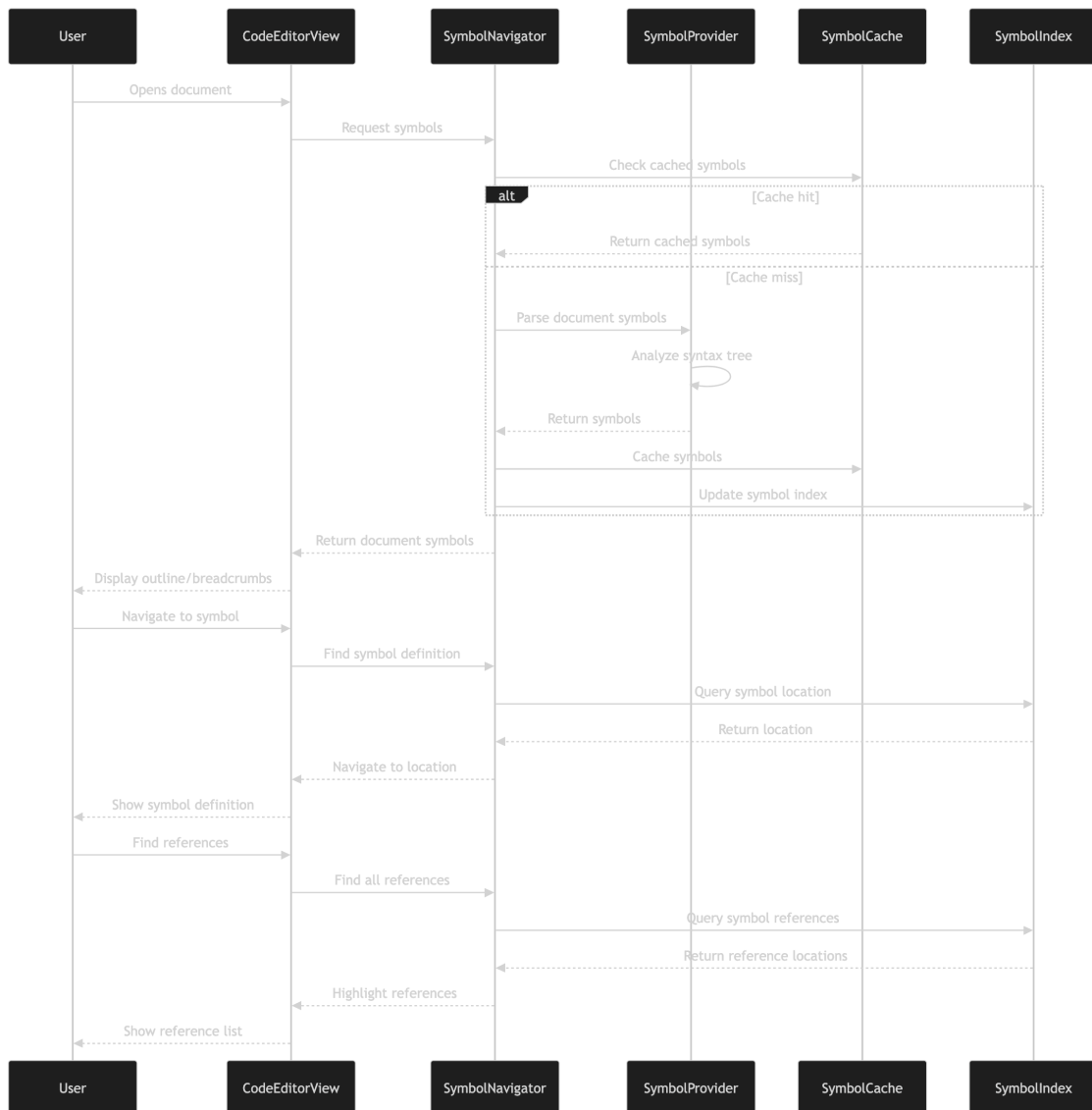
1. **Consistent Performance:** Maintains 60fps across all operations
2. **Memory Efficiency:** Handles large files without memory bloat
3. **Adaptive Behavior:** Automatically optimizes for current conditions
4. **Proactive Optimization:** Prevents performance issues before they occur
5. **Data-Driven:** Uses metrics to make optimization decisions

14 Symbol Navigation & Code Intelligence

This diagram shows the comprehensive symbol navigation and code intelligence system that provides advanced code understanding and navigation capabilities.



14.1 Symbol Navigation Flow



14.2 Key Symbol Intelligence Features

14.2.1.1. Multi-Language Symbol Providers

- **Swift Integration:** Full SwiftSyntax AST analysis
- **Python Support:** AST-based symbol extraction with import resolution
- **JavaScript/TypeScript:** Babel parser with type inference
- **Generic Fallback:** Pattern-based extraction for any language

14.2.2 2. Document Outline System

- **Hierarchical View:** Nested symbol structure display
- **Filtering & Search:** Real-time symbol filtering
- **Expandable Nodes:** Interactive tree navigation
- **Symbol Grouping:** Organize by type, visibility, or custom rules

14.2.3 3. Breadcrumb Navigation

- **Scope Awareness:** Shows current code context
- **Interactive Navigation:** Click to navigate to any scope level
- **Context Menus:** Quick actions for each breadcrumb item
- **Real-time Updates:** Updates as cursor moves

14.2.4 4. Reference & Definition Resolution

- **Go-to-Definition:** Precise symbol definition location
- **Find All References:** Complete symbol usage analysis
- **Cross-file Navigation:** Multi-document symbol resolution
- **Import Resolution:** Automatic import path resolution

14.2.5 5. Optimized Performance

- **Background Indexing:** Non-blocking symbol parsing
- **Incremental Updates:** Efficient partial re-indexing
- **Smart Caching:** LRU cache with persistent storage
- **Query Optimization:** Fast symbol search and lookup

14.2.6 6. Advanced Analysis

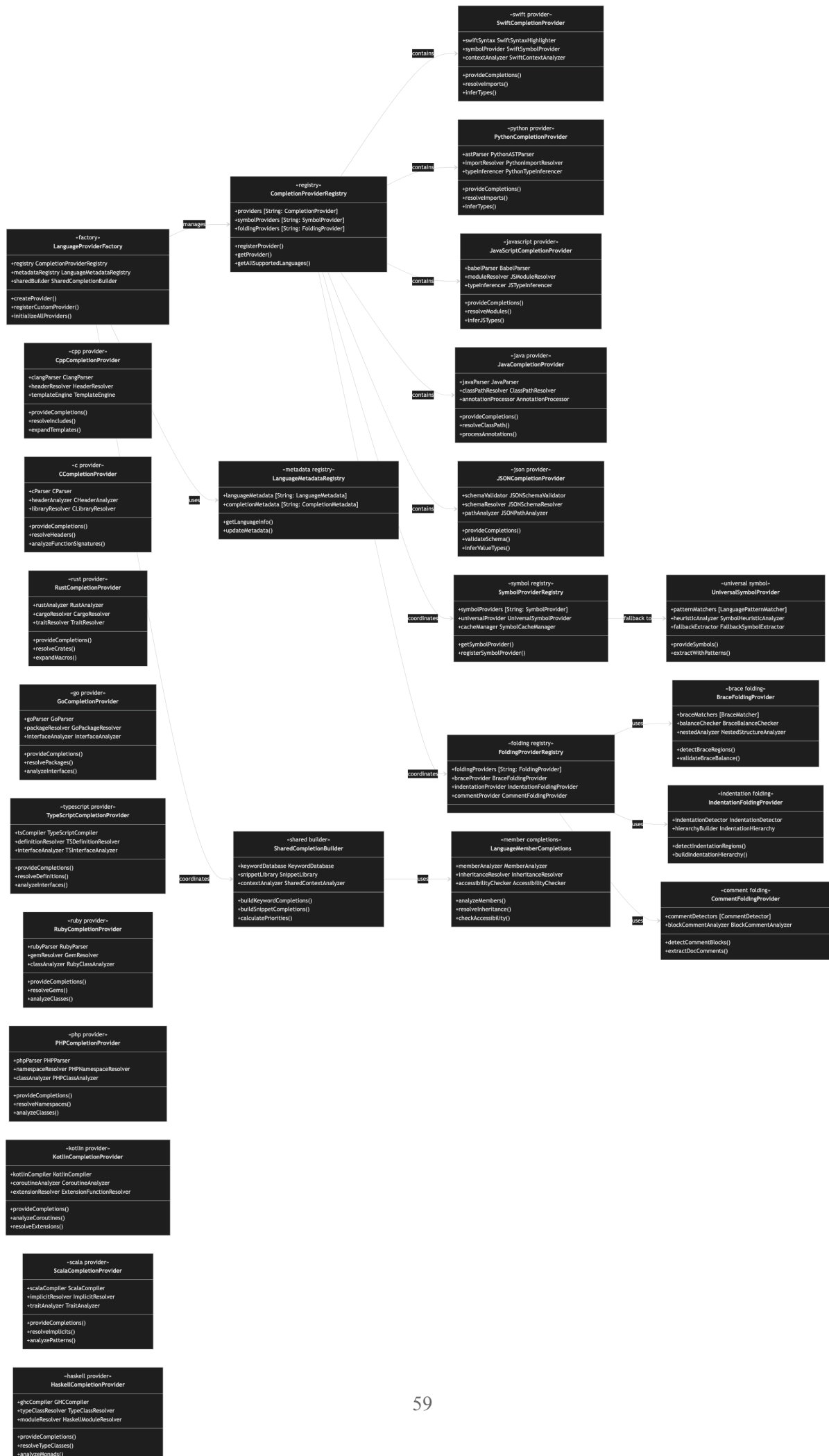
- **Scope Analysis:** Understand symbol visibility and context
- **Dependency Tracking:** Symbol relationship mapping
- **Circular Reference Detection:** Identify problematic dependencies
- **Usage Analytics:** Track symbol access patterns

14.3 Benefits

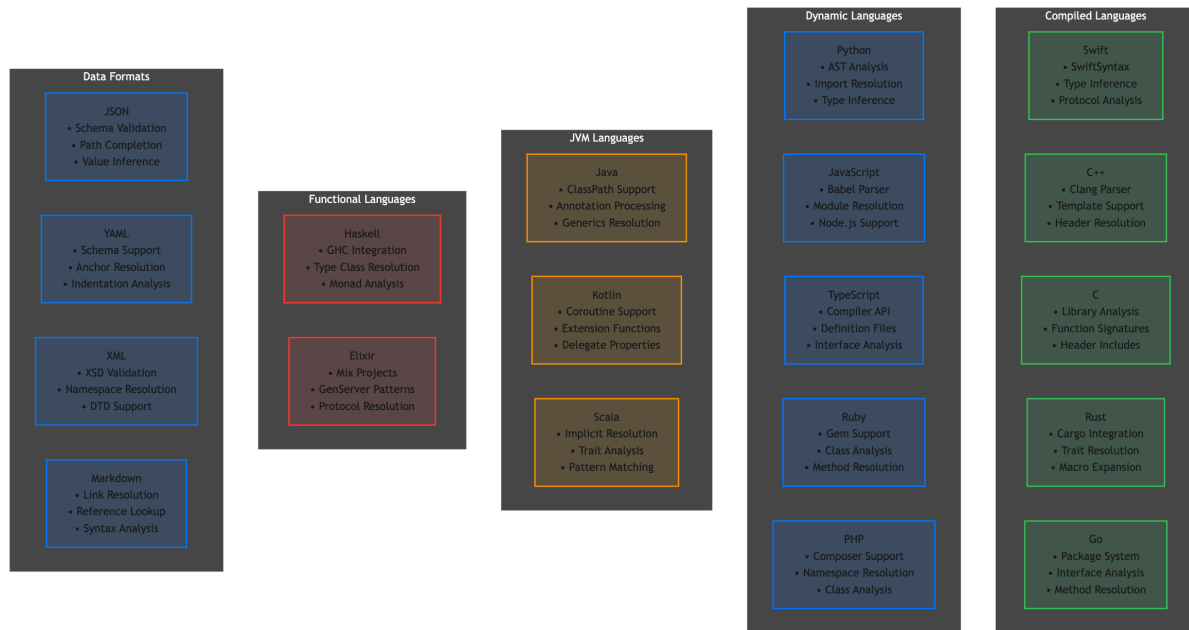
1. **Fast Navigation:** Instant symbol lookup and navigation
2. **Language Agnostic:** Works across all supported languages
3. **Context Aware:** Understands code structure and relationships
4. **Scalable:** Handles large codebases efficiently
5. **Extensible:** Plugin architecture for new languages
6. **Persistent:** Maintains symbol information across sessions

15 Language Provider Complete Ecosystem

This comprehensive diagram shows the complete language provider ecosystem supporting 17+ languages with completion, symbols, folding, and data providers.



15.1 Language Support Matrix



15.2 Key Ecosystem Features

15.2.1 1. Universal Provider Factory

- **Automatic Registration:** Self-registering language providers
- **Capability Discovery:** Runtime provider capability detection
- **Dynamic Loading:** Lazy loading of language-specific providers
- **Custom Provider Support:** Plugin architecture for additional languages

15.2.2 2. Comprehensive Language Support

- **17+ Languages:** Full support for major programming languages
- **Data Format Support:** JSON, YAML, XML, Markdown completion
- **Domain-Specific:** Specialized providers for different language paradigms
- **Extensible Architecture:** Easy addition of new language providers

15.2.3 3. Shared Infrastructure

- **Common Completion Logic:** Reusable completion building components
- **Keyword Databases:** Centralized keyword management
- **Snippet Libraries:** Shared code snippet repositories
- **Context Analysis:** Universal context understanding

15.2.4 4. Advanced Features

- **Symbol Resolution:** Cross-file symbol navigation
- **Import/Module Resolution:** Automatic dependency resolution
- **Type Inference:** Intelligent type analysis where applicable
- **Documentation Integration:** Inline documentation support

15.2.5 5. Performance Optimizations

- **Lazy Loading:** Providers loaded on demand
- **Caching:** Intelligent caching of parsing results
- **Background Processing:** Non-blocking completion generation
- **Incremental Parsing:** Efficient re-parsing on changes

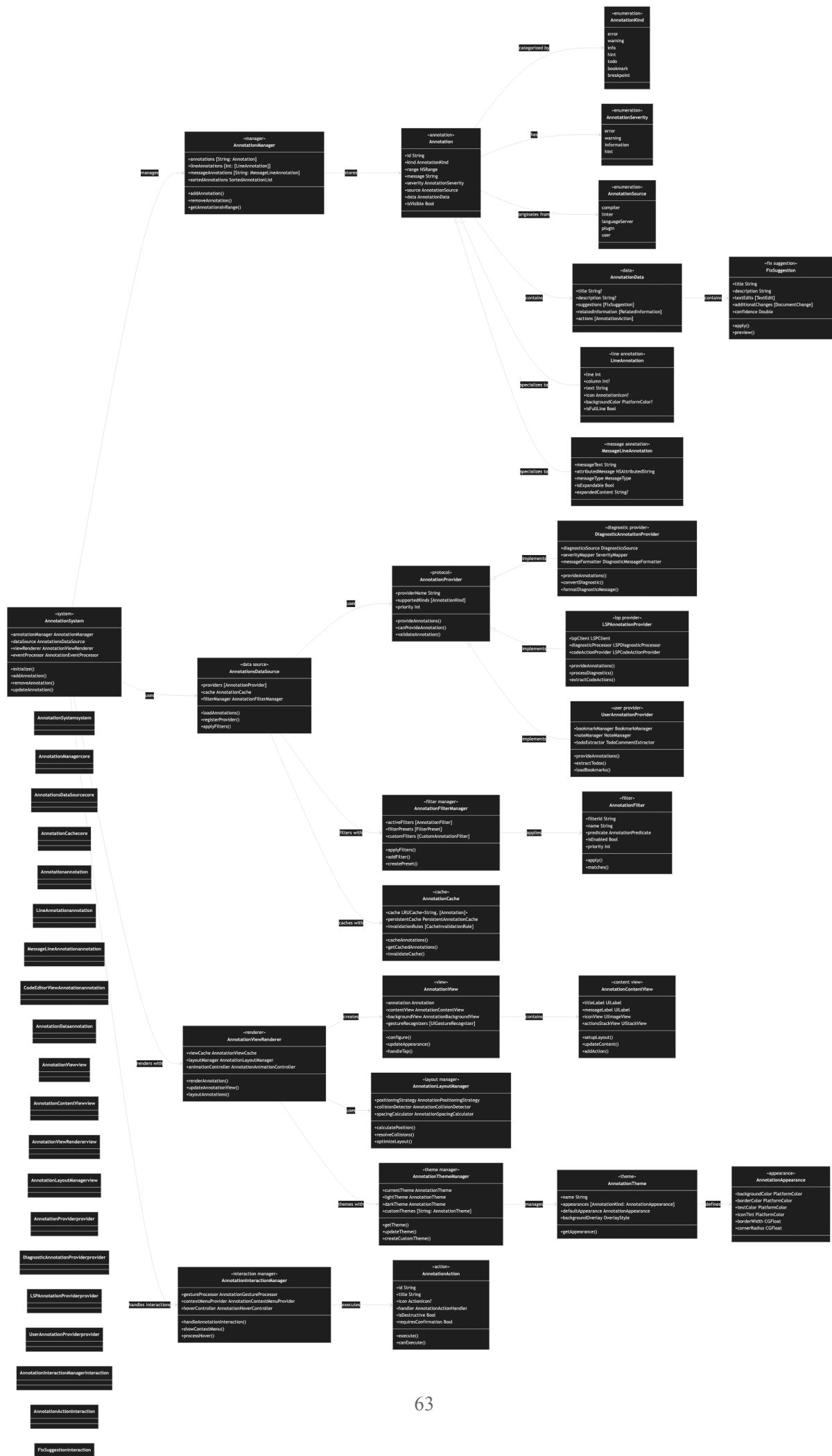
15.3 Benefits

1. **Comprehensive Coverage:** Support for virtually any programming language
2. **Consistent Experience:** Uniform completion behavior across languages
3. **High Performance:** Optimized for speed and responsiveness
4. **Extensible:** Easy to add support for new languages
5. **Maintainable:** Shared infrastructure reduces code duplication
6. **Scalable:** Handles large codebases and complex projects efficiently

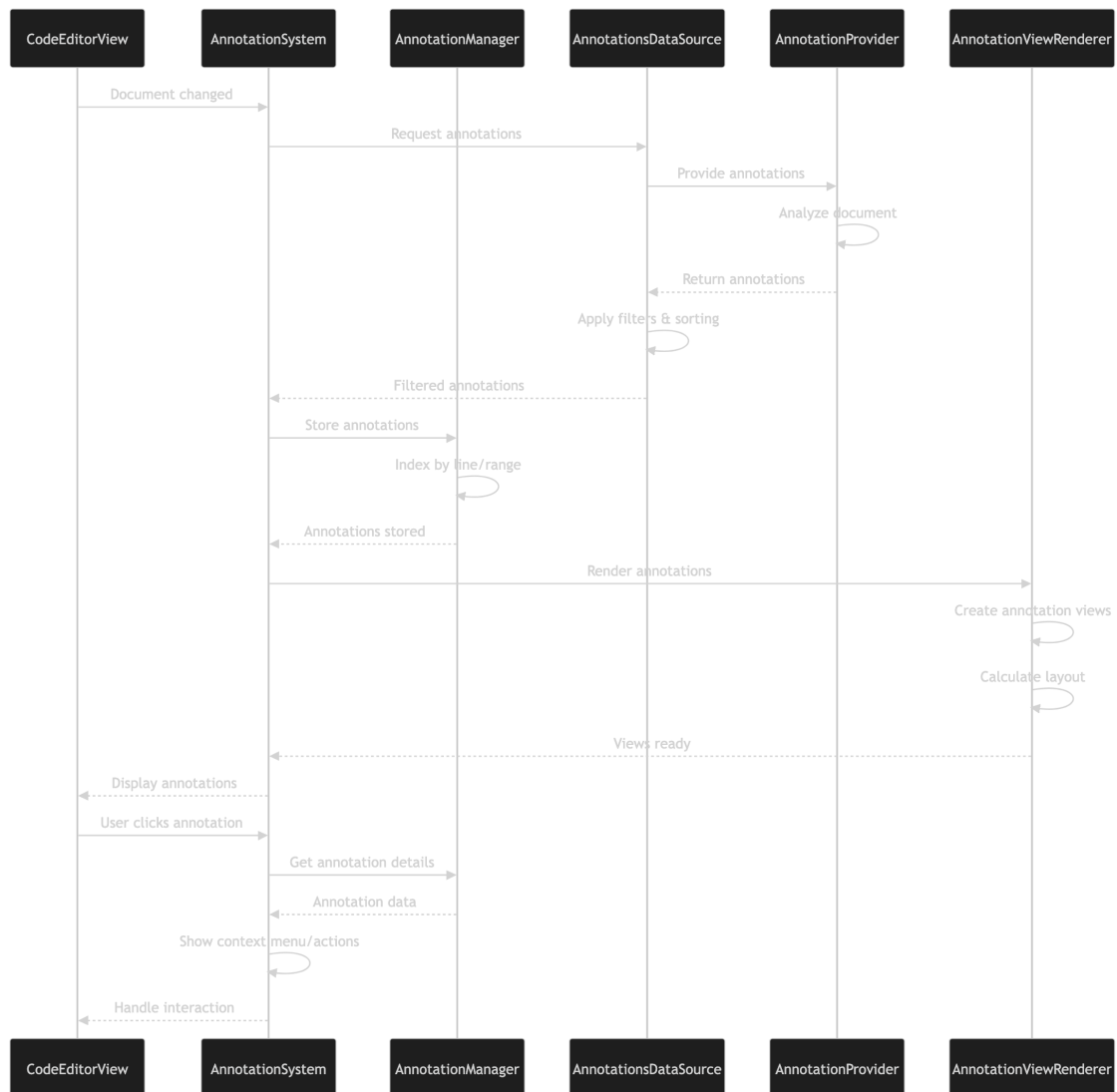
16 Annotation System Detailed Architecture

This diagram shows the comprehensive annotation system that provides code annotations, diagnostics, and contextual information overlay capabilities.

16 Annotation System Detailed Architecture



16.1 Annotation System Flow



16.2 Key Annotation Features

16.2.1 1. Multi-Source Annotation Support

- **Diagnostic Annotations:** Compiler errors, warnings, and hints
- **LSP Integration:** Language server diagnostics and code actions
- **User Annotations:** Bookmarks, notes, and custom markers
- **Plugin Support:** Third-party annotation providers

16.2.2 2. Rich Annotation Types

- **Line Annotations:** Simple line-based markers and messages
- **Message Annotations:** Detailed messages with formatting
- **Code Editor Annotations:** Complex overlay annotations
- **Contextual Information:** Related information and suggestions

16.2.3 3. Interactive Features

- **Click Actions:** Execute fixes and navigate to related code
- **Hover Information:** Show detailed information on hover
- **Context Menus:** Quick actions and options
- **Keyboard Navigation:** Navigate between annotations

16.2.4 4. Smart Filtering and Organization

- **Filter by Type:** Error, warning, info, custom categories
- **Filter by Source:** Compiler, LSP, user, plugins
- **Custom Filters:** User-defined filtering criteria
- **Sorting Options:** By severity, line, timestamp, source

16.2.5 5. Visual Customization

- **Theme Support:** Light/dark mode with custom themes
- **Appearance Configuration:** Colors, borders, shadows
- **Animation Support:** Smooth transitions and effects
- **Layout Options:** Positioning and collision resolution

16.2.6 6. Performance Optimizations

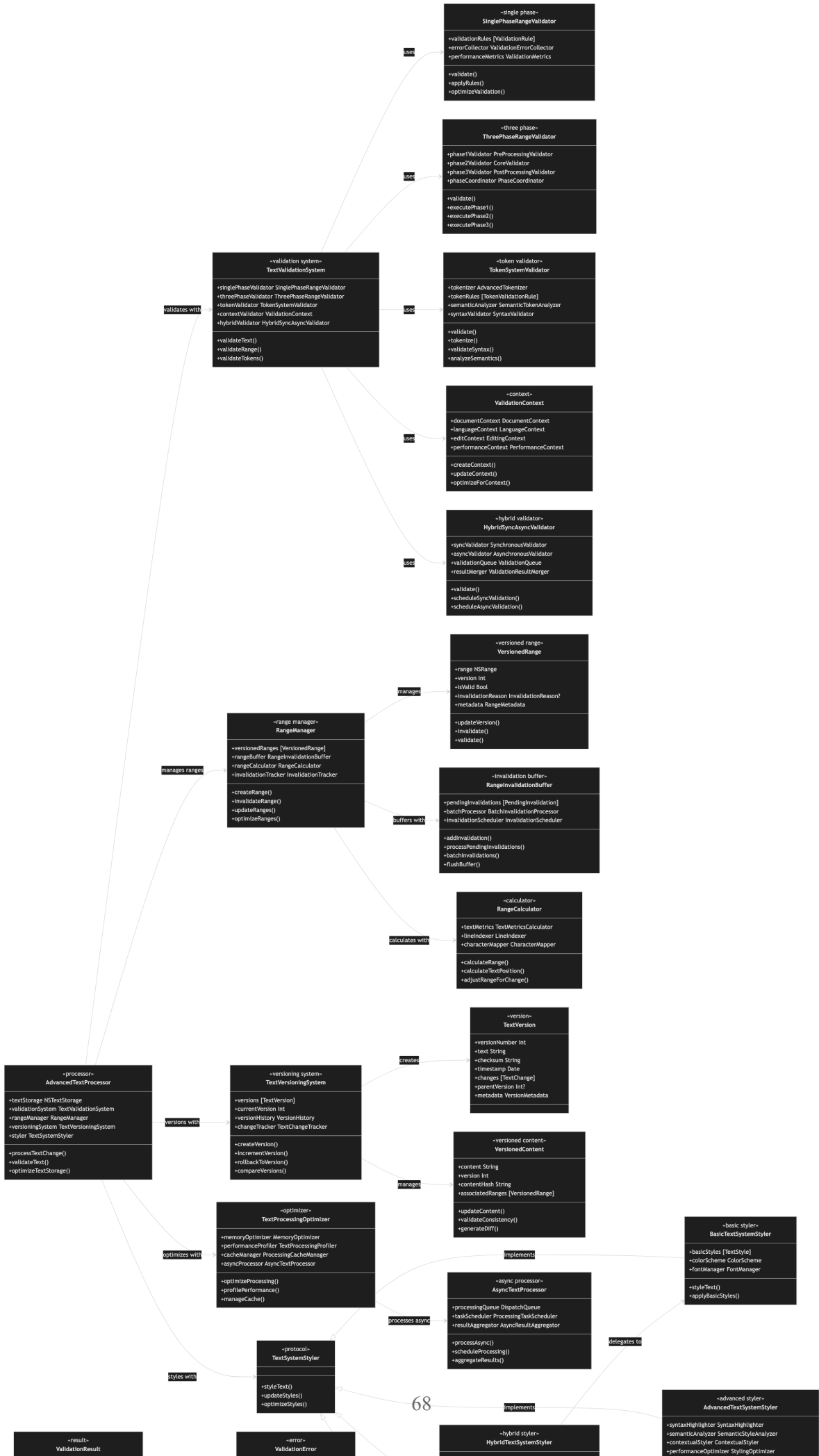
- **View Recycling:** Efficient view reuse for large documents
- **Incremental Updates:** Only update changed annotations
- **Lazy Loading:** Load annotations as needed
- **Caching:** Cache annotation data and views

16.3 Benefits

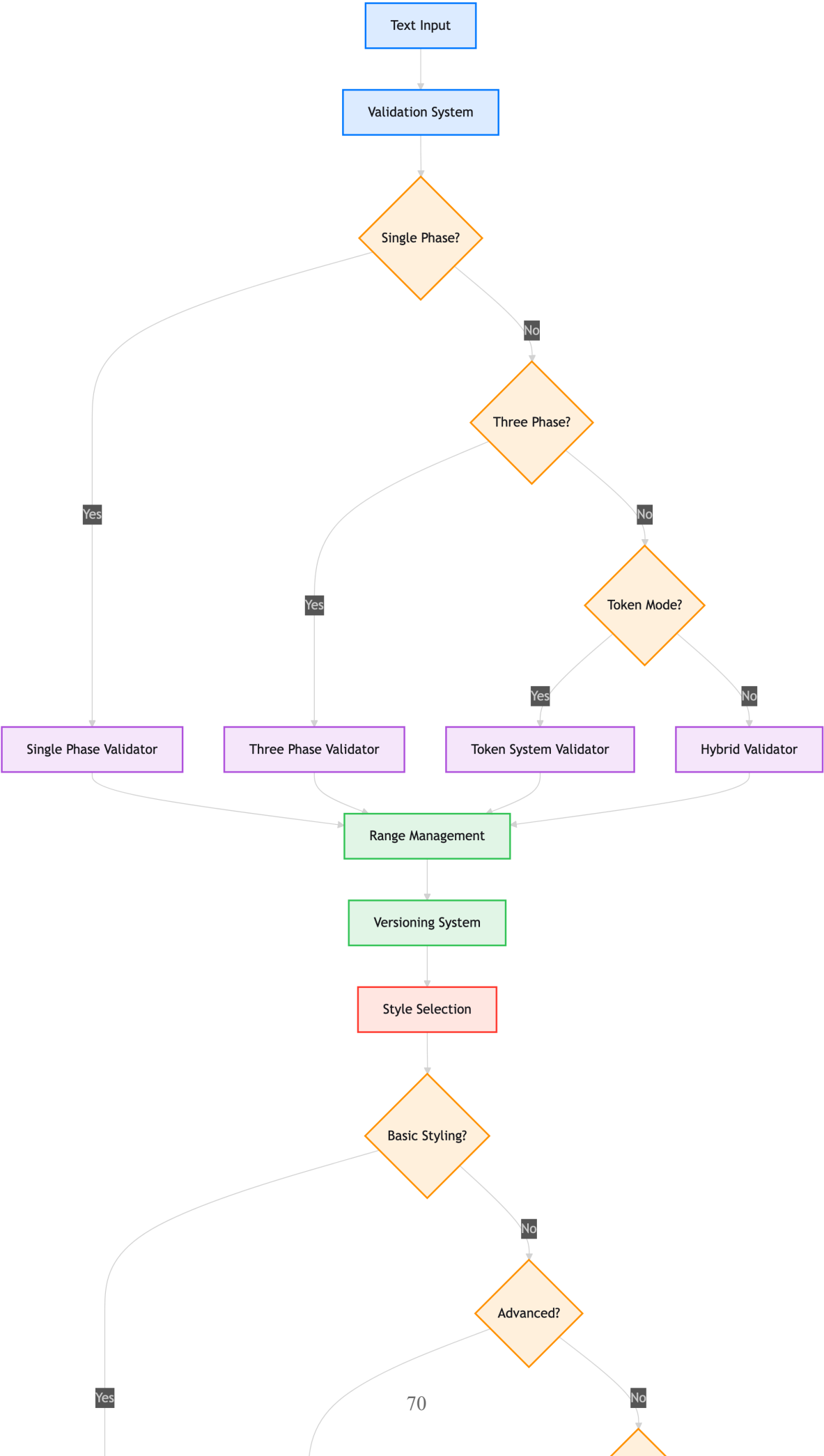
1. **Comprehensive Feedback:** Multi-source annotation integration
2. **Rich Interactivity:** Clickable actions and contextual information
3. **Customizable:** Extensive theming and filtering options
4. **Performance:** Optimized for large documents with many annotations
5. **Extensible:** Plugin architecture for custom annotation types
6. **User-Friendly:** Intuitive interaction patterns and visual design

17 Advanced Text Processing & Validation Pipeline

This diagram shows the comprehensive text processing and validation system that ensures robust text handling, range validation, and performance optimization.



17.1 Text Processing Flow



17.2 Key Text Processing Features

17.2.1 1. Multi-Phase Validation System

- **Single Phase:** Fast validation for simple text changes
- **Three Phase:** Comprehensive validation with pre/core/post processing
- **Token-Based:** Advanced syntax and semantic validation
- **Hybrid Sync/Async:** Balanced performance and thoroughness

17.2.2 2. Advanced Range Management

- **Versioned Ranges:** Track range validity across text changes
- **Invalidation Buffer:** Efficient batch processing of range updates
- **Smart Recalculation:** Optimize range updates for performance
- **Metadata Tracking:** Associate custom data with text ranges

17.2.3 3. Text Versioning System

- **Version History:** Complete change tracking and rollback capability
- **Content Integrity:** Checksum validation and consistency checking
- **Change Tracking:** Detailed change metadata and analysis
- **Diff Generation:** Efficient difference calculation between versions

17.2.4 4. Flexible Styling System

- **Multiple Styler Types:** Basic, advanced, optimized, and hybrid stylers
- **Contextual Styling:** Style based on document context and language
- **Performance Optimization:** Incremental styling and background processing
- **Cache Management:** Efficient style result caching and invalidation

17.2.5 5. Performance Optimization

- **Memory Management:** Efficient memory usage and cleanup

- **Async Processing:** Non-blocking text processing operations
- **Smart Caching:** Intelligent caching of processing results
- **Performance Profiling:** Built-in performance monitoring and optimization

17.2.6 6. Comprehensive Validation

- **Multi-Level Errors:** Errors, warnings, hints, and suggestions
- **Context-Aware:** Validation based on document and language context
- **Fix Suggestions:** Automatic fix recommendations
- **Performance Metrics:** Validation performance tracking

17.3 Benefits

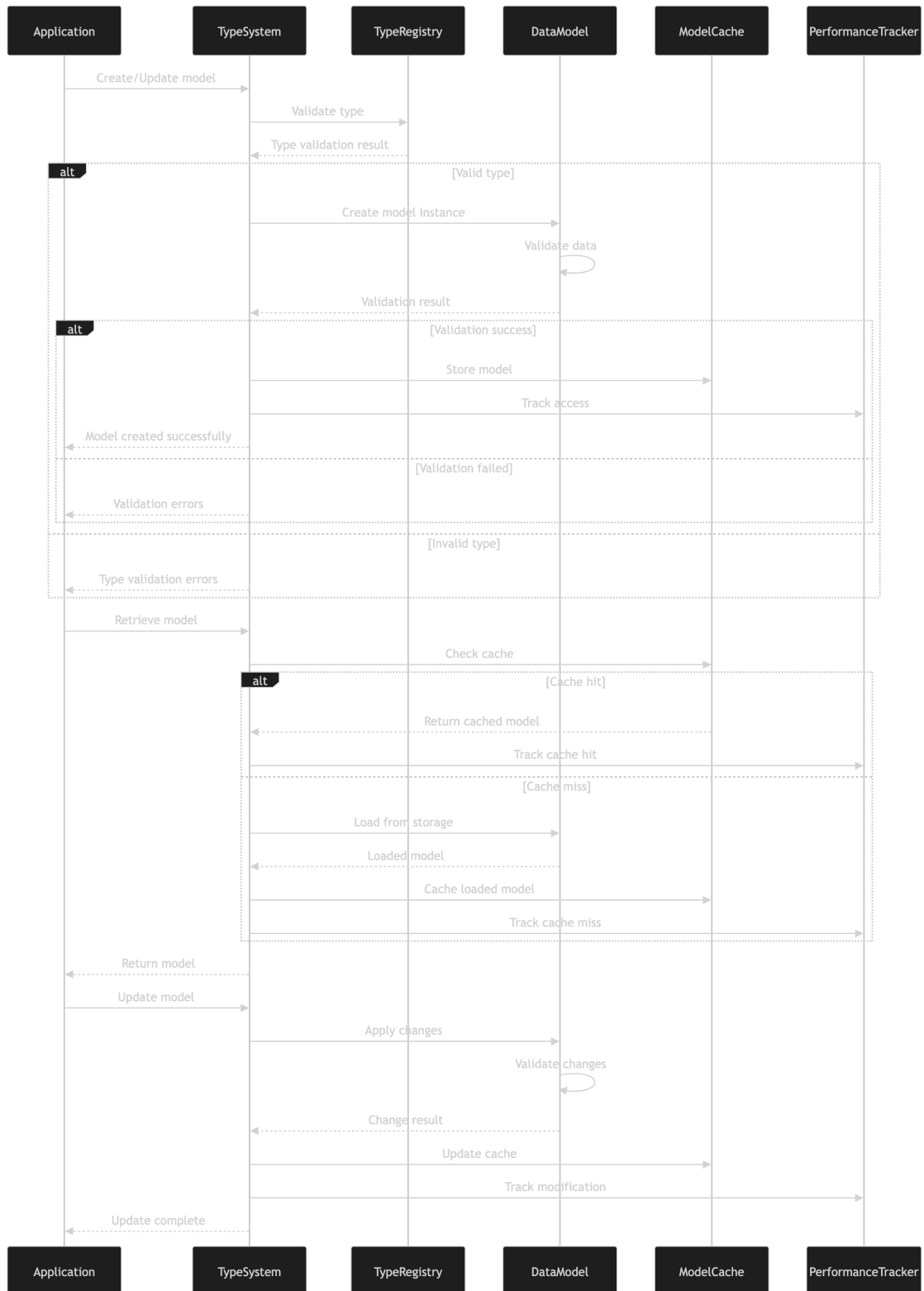
1. **Robust Text Handling:** Comprehensive validation and error handling
2. **High Performance:** Optimized for large documents and frequent changes
3. **Flexible Architecture:** Multiple validation and styling strategies
4. **Version Control:** Complete text history and change tracking
5. **Extensible:** Plugin architecture for custom validators and stylers
6. **Memory Efficient:** Smart caching and memory management

18 Data Models & Type System Architecture

This diagram shows the comprehensive data models and type system that forms the foundation of the CodeEditorPlugin's data structures and type safety.



18.1 Data Model Interaction Flow



18.2 Key Data Model Features

18.2.1 1. Comprehensive Type System

- **Type Registry:** Central type registration and validation
- **Type Inference:** Automatic type detection and inference
- **Generic Support:** Full generic type parameter support
- **Constraint System:** Type constraints and validation rules

18.2.2 2. Rich Text Models

- **Foldable Regions:** Hierarchical code folding with metadata
- **Marked Text:** Text with multiple markers and annotations
- **Token System:** Comprehensive tokenization with semantic info
- **Text Segments:** Typed text segments with attributes

18.2.3 3. Versioning and History

- **Generic Versioning:** Version any data type with full history
- **Content Management:** Rich content with encoding and metadata
- **Difference Tracking:** Detailed change analysis and patch generation
- **Rollback Support:** Full version rollback capabilities

18.2.4 4. Range and Mutation System

- **Range Mutations:** Comprehensive text range modification tracking
- **Mutation Types:** Full spectrum of text change operations
- **Reversible Operations:** Undo/redo support for all mutations
- **Batch Operations:** Efficient handling of multiple changes

18.2.5 5. Advanced Type Information

- **Rich Type Metadata:** Comprehensive type information with documentation
- **Generic Parameters:** Full generic type support with constraints

- **Member Information:** Detailed type member analysis
- **Inheritance Tracking:** Complete inheritance hierarchy support

18.2.6 6. Performance Optimization

- **Model Caching:** Efficient caching with LRU and validation
- **Performance Tracking:** Comprehensive performance monitoring
- **Access Pattern Analysis:** Optimize based on usage patterns
- **Memory Management:** Smart memory usage and cleanup

18.2.7 7. Content Type Support

- **Multiple Formats:** Support for various content types and encodings
- **Line Ending Handling:** Comprehensive line ending support
- **Binary Content:** Support for both text and binary content
- **MIME Type Integration:** Standard MIME type recognition

18.3 Benefits

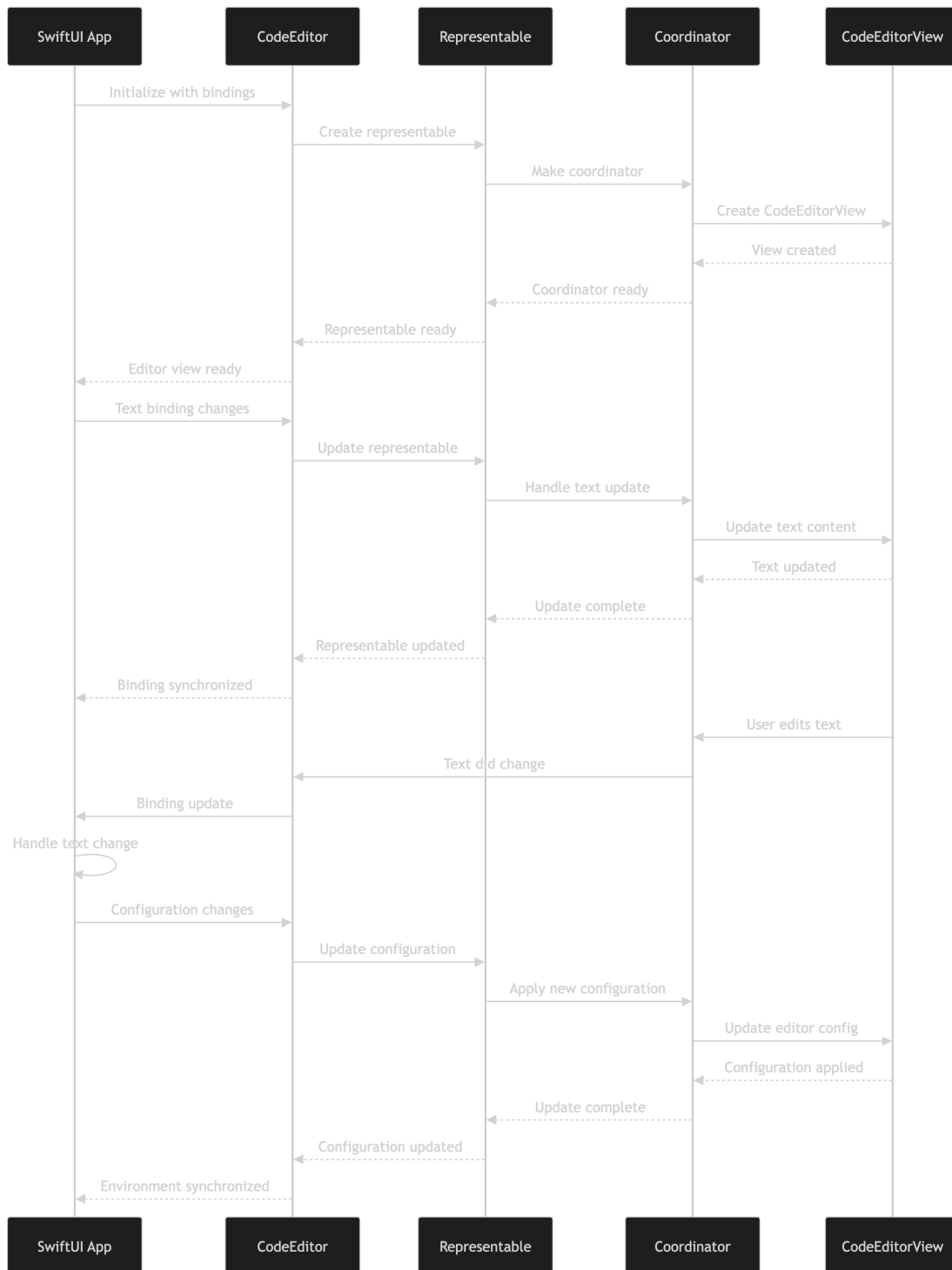
1. **Type Safety:** Comprehensive type validation and inference
2. **Rich Metadata:** Detailed information for all data models
3. **Version Control:** Complete history and change tracking
4. **Performance:** Optimized caching and access patterns
5. **Extensibility:** Easy addition of new data types and models
6. **Consistency:** Uniform data model patterns throughout framework

19 SwiftUI Integration Complete Ecosystem

This diagram shows the comprehensive SwiftUI integration ecosystem that provides seamless integration between the CodeEditorPlugin framework and SwiftUI applications.



19.1 SwiftUI Integration Flow



19.2 Key SwiftUI Integration Features

19.2.1 1. Seamless SwiftUI Integration

- **Native SwiftUI View:** CodeEditor acts as a true SwiftUI view
- **Binding Support:** Full two-way binding with SwiftUI state
- **Environment Integration:** Uses SwiftUI environment system
- **Modifier Support:** Custom view modifiers for configuration

19.2.2 2. Platform-Specific Representables

- **AppKit Integration:** Native macOS NSViewRepresentable
- **UIKit Integration:** Native iOS/iPadOS UIViewRepresentable
- **Mac Catalyst:** Specialized Catalyst representable
- **Cross-Platform Coordination:** Unified behavior across platforms

19.2.3 3. Advanced State Management

- **Binding Synchronization:** Automatic sync between SwiftUI and CodeEditor
- **State Validation:** Comprehensive state validation and consistency
- **Change Observation:** Reactive updates to state changes
- **Version Tracking:** State versioning and history

19.2.4 4. Rich Environment System

- **Environment Values:** Custom environment values for configuration
- **Theme Integration:** Automatic theme propagation through environment
- **Language Support:** Language configuration via environment
- **Accessibility:** Full accessibility environment integration

19.2.5 5. View Modifier Ecosystem

- **Fluent API:** Chainable view modifiers for configuration
- **Custom Modifiers:** Extensible modifier system

- **Priority System:** Modifier application order management
- **Animation Support:** Smooth transitions for modifier changes

19.2.6 6. Animation and Transitions

- **SwiftUI Animations:** Native SwiftUI animation support
- **Custom Transitions:** Specialized code editor transitions
- **Performance Optimized:** Animations optimized for text editing
- **Accessibility Aware:** Respects reduce motion preferences

19.2.7 7. Development Support

- **Preview Support:** Comprehensive SwiftUI preview integration
- **Mock Data:** Rich mock data for development and testing
- **Debug Mode:** Special debug overlays and information
- **Extension System:** Plugin architecture for custom SwiftUI components

19.3 Usage Examples

19.3.1 Basic Implementation

```
struct ContentView: View {
    @State private var code = "// Hello World"
    @State private var config = EditorConfiguration.default

    var body: some View {
        CodeEditor(text: $code)
            .codeLanguage(.swift)
            .codeTheme(.xcode)
            .environment(\.codeEditorConfiguration, config)
    }
}
```

19.3.2 Advanced Configuration

```
CodeEditor(text: $sourceCode)
    .codeLanguage(.python)
    .codeTheme(.github)
    .readOnly(isReadOnly)
    .showLineNumbers(true)
    .enableSyntaxHighlighting(true)
    .onTextChange { newText in
        handleTextChange(newText)
    }
    .onSelectionChange { range in
        handleSelectionChange(range)
    }
}
```

19.3.3 Custom Environment

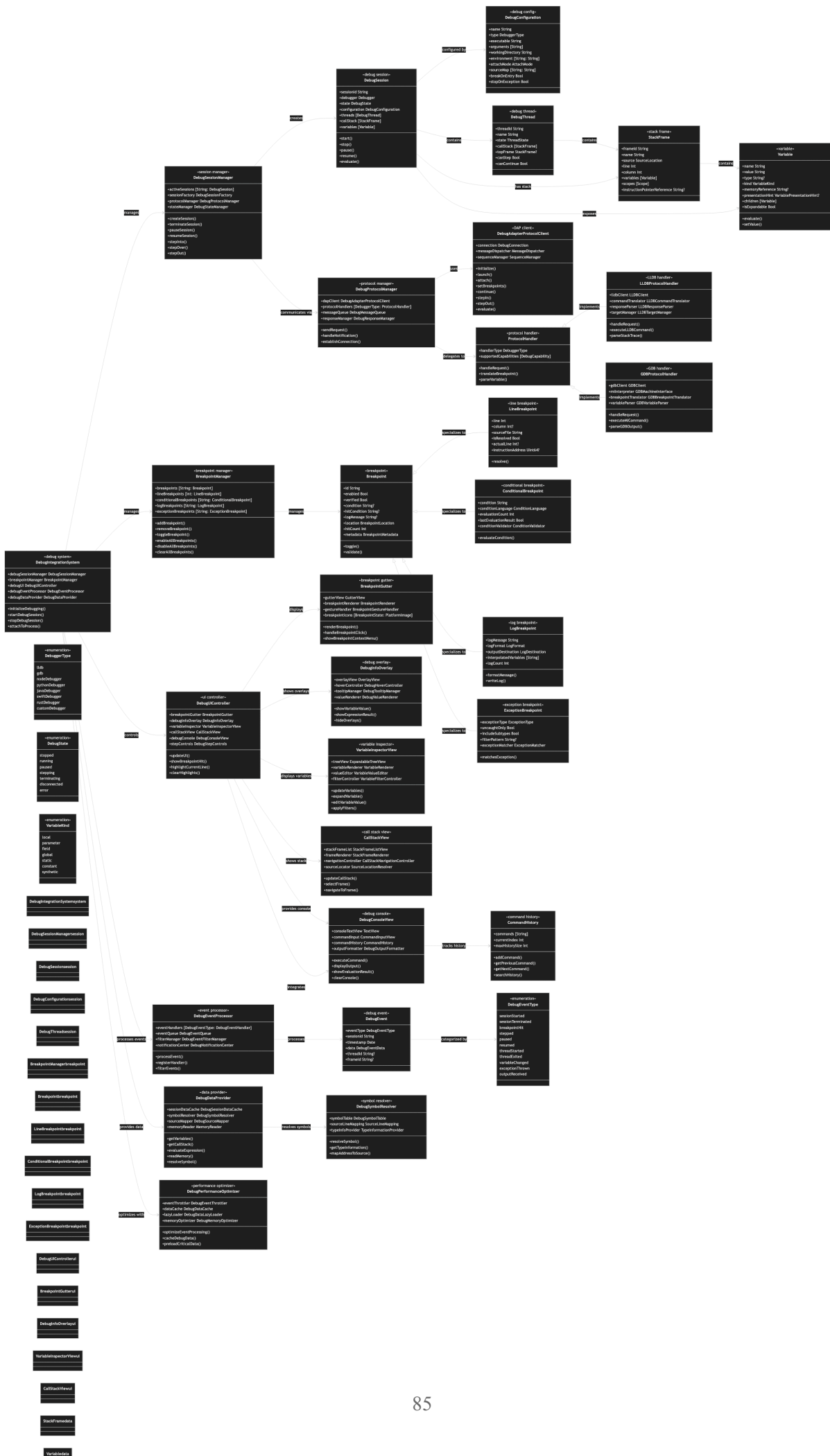
```
CodeEditor(text: $code)
    .environment(\.codeEditorConfiguration, customConfig)
    .environment(\.codeEditorTheme, darkTheme)
    .environment(\.codeEditorLanguage, .javascript)
```

19.4 Benefits

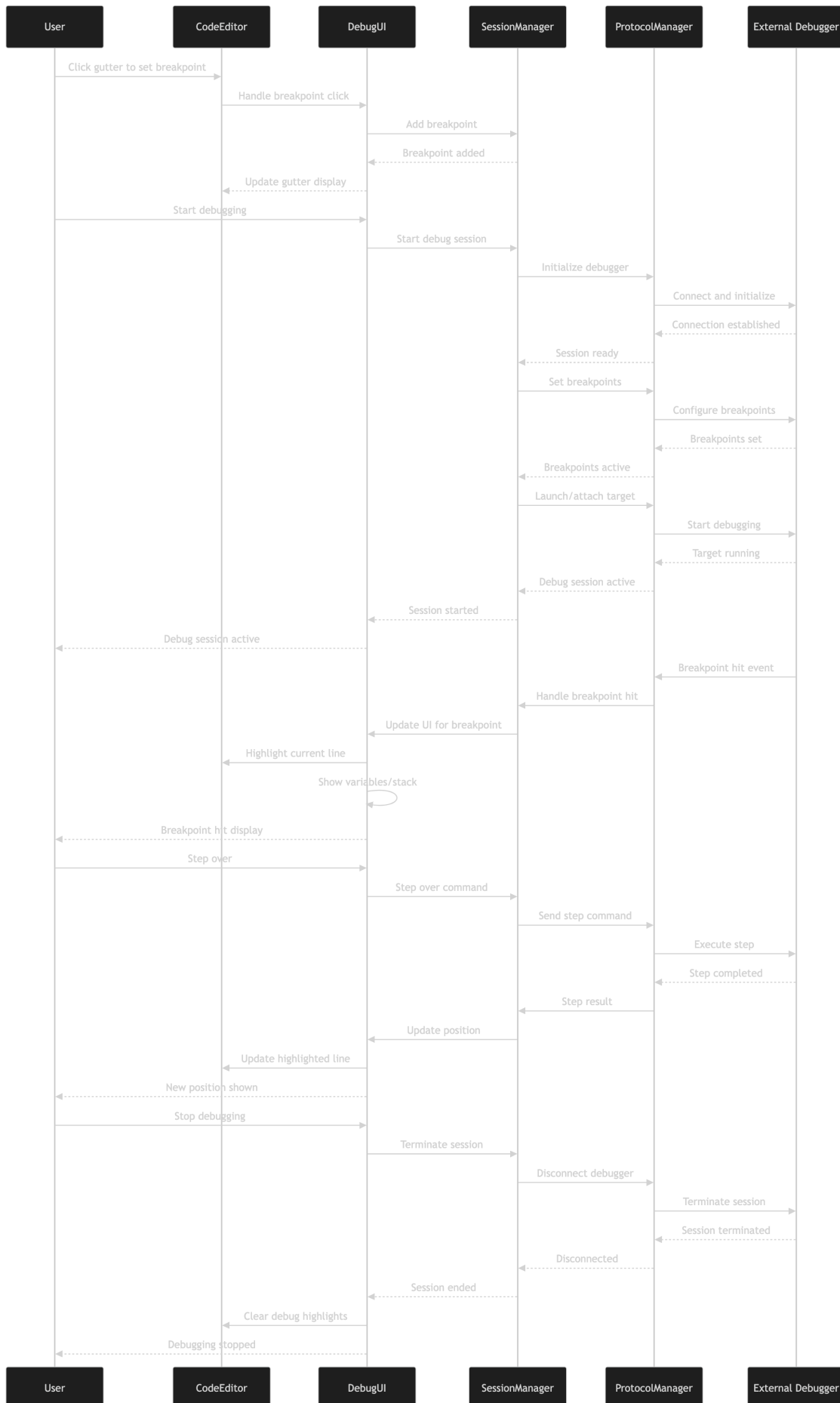
1. **Native SwiftUI:** True SwiftUI integration with full ecosystem support
2. **Cross-Platform:** Single API works across macOS, iOS, and Catalyst
3. **Reactive:** Automatic synchronization with SwiftUI state management
4. **Extensible:** Rich modifier and extension system
5. **Accessible:** Full accessibility support with VoiceOver integration
6. **Developer Friendly:** Comprehensive preview and debugging support

20 Debugging Integration Detailed Architecture

This diagram shows the comprehensive debugging integration system that provides breakpoint management, debug session control, and debugging visualization capabilities within the code editor.



20.1 Debug Integration Flow



20.2 Key Debugging Features

20.2.1 1. Multi-Debugger Support

- **LLDB Integration:** Native debugging for Swift, C, C++, Objective-C
- **GDB Support:** GNU debugger for C, C++, and other compiled languages
- **Language-Specific:** Specialized debuggers for Python, Node.js, Java
- **Debug Adapter Protocol:** Standard protocol for debugger integration

20.2.2 2. Comprehensive Breakpoint Management

- **Line Breakpoints:** Simple line-based breakpoints
- **Conditional Breakpoints:** Break only when conditions are met
- **Log Breakpoints:** Output messages without stopping execution
- **Exception Breakpoints:** Break on specific exception types

20.2.3 3. Rich Debug UI

- **Breakpoint Gutter:** Visual breakpoint indicators in editor
- **Variable Inspector:** Hierarchical variable viewing and editing
- **Call Stack View:** Navigate through execution stack frames
- **Debug Console:** Interactive command execution and output

20.2.4 4. Advanced Debug Features

- **Expression Evaluation:** Evaluate expressions in debug context
- **Memory Inspection:** View and modify memory contents
- **Symbol Resolution:** Navigate to symbol definitions
- **Source Mapping:** Handle source maps and transformed code

20.2.5 5. Interactive Debugging

- **Step Controls:** Step in, over, out, and continue operations
- **Hover Inspection:** View variable values on hover

- **Context Menus:** Quick actions for breakpoints and variables
- **Keyboard Shortcuts:** Efficient debugging workflow

20.2.6 6. Performance Optimizations

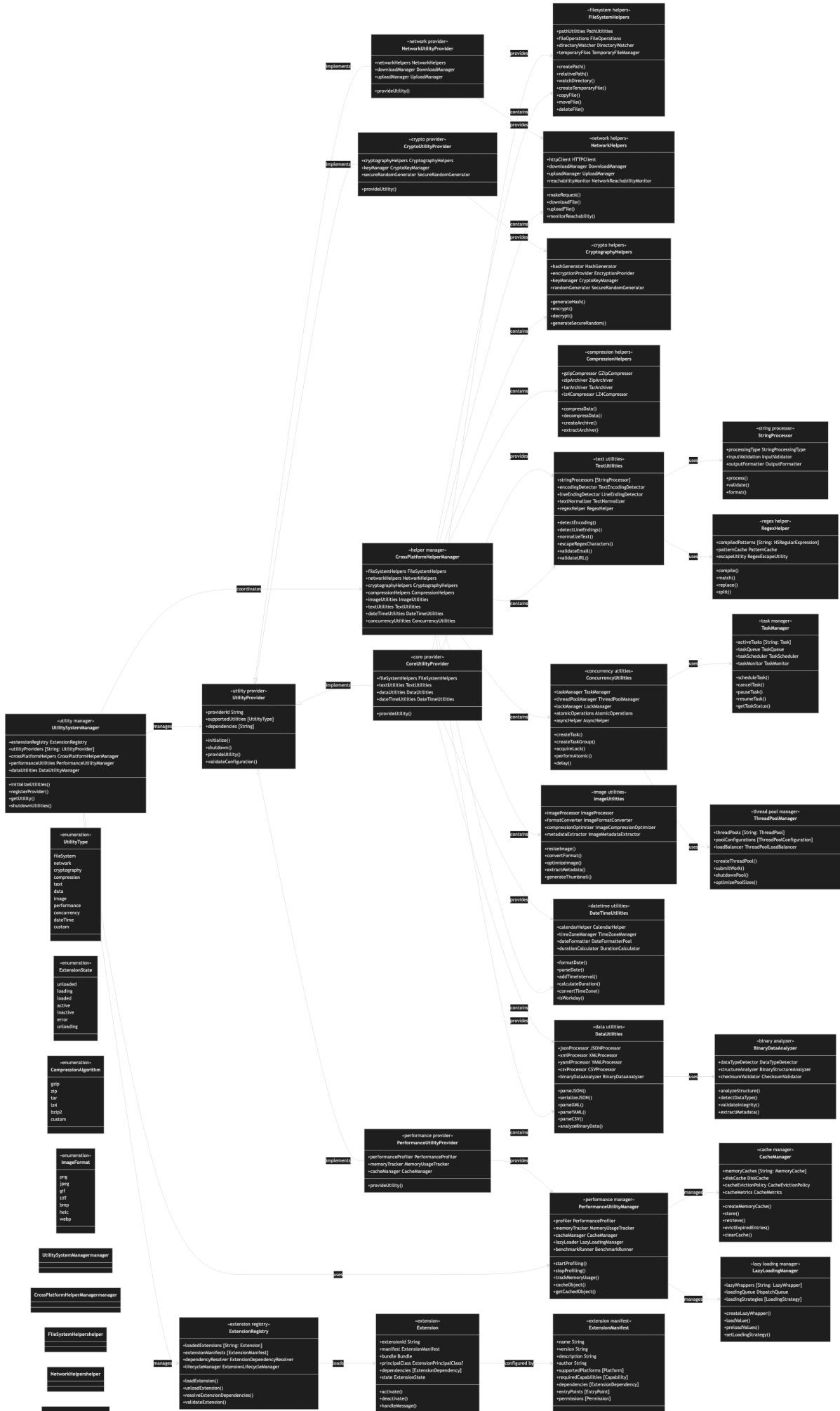
- **Event Throttling:** Prevent UI flooding from rapid debug events
- **Lazy Loading:** Load debug data on demand
- **Data Caching:** Cache frequently accessed debug information
- **Memory Management:** Efficient cleanup of debug sessions

20.3 Benefits

1. **Universal Debugging:** Support for multiple debuggers and languages
2. **Rich Visualization:** Comprehensive debug information display
3. **Interactive Experience:** Intuitive debugging workflow
4. **Performance:** Optimized for responsive debugging sessions
5. **Extensible:** Plugin architecture for additional debugger types
6. **Standards Compliant:** Debug Adapter Protocol support for compatibility

21 Utility Systems & Extensions Network

This diagram shows the comprehensive utility systems and extensions network that provides shared utilities, cross-platform helpers, and extensibility infrastructure throughout the CodeEditorPlugin framework.



21.1 Utility System Integration Flow



21.2 Key Utility System Features

21.2.1. Comprehensive Platform Utilities

- **File System Helpers:** Cross-platform file operations, path utilities, directory watching
- **Network Helpers:** HTTP client, download/upload management, reachability monitoring
- **Cryptography Helpers:** Secure hashing, encryption/decryption, key management
- **Compression Helpers:** Multiple compression algorithms with archive support

21.2.2. Advanced Data Processing

- **Text Utilities:** Encoding detection, normalization, regex helpers, validation
- **Data Utilities:** JSON/XML/YAML/CSV processing, binary data analysis
- **Image Utilities:** Format conversion, compression optimization, metadata extraction
- **Binary Analysis:** Data type detection, structure analysis, integrity validation

21.2.3 3. Performance Optimization

- **Cache Management:** Multi-level caching with eviction policies
- **Lazy Loading:** Smart loading strategies with preload capabilities
- **Performance Profiling:** Built-in profiling and benchmarking tools
- **Memory Tracking:** Comprehensive memory usage monitoring

21.2.4 4. Concurrency Support

- **Task Management:** Comprehensive task scheduling and monitoring
- **Thread Pool Management:** Dynamic thread pool optimization
- **Lock Management:** Advanced locking primitives and strategies
- **Atomic Operations:** Thread-safe atomic operations

21.2.5 5. Extension System

- **Dynamic Loading:** Runtime extension loading and unloading
- **Dependency Resolution:** Automatic dependency management
- **Lifecycle Management:** Complete extension lifecycle control
- **Validation:** Extension validation and security checks

21.2.6 6. Date and Time Support

- **Calendar Helpers:** Advanced calendar operations and calculations
- **Time Zone Management:** Cross-timezone date/time handling
- **Date Formatting:** Flexible date formatting with locale support
- **Duration Calculations:** Precise duration and interval calculations

21.3 Usage Examples

21.3.1 File System Operations

```
let fileHelpers =
    UtilitySystemManager.shared.getUtility(FileSystemHelpers.self)
let tempFile = fileHelpers?.createTemporaryFile(prefix:
    "editor_cache")
let success = fileHelpers?.copyFile(from: sourcePath, to:
    destPath)
```

21.3.2 Network Operations

```
let networkHelpers =
    UtilitySystemManager.shared.getUtility(NetworkHelpers.self)
let response = await networkHelpers?.makeRequest(httpRequest)
let downloadTask = networkHelpers?.downloadFile(url: url,
    destination: path)
```

21.3.3 Performance Monitoring

```
let perfManager =
    UtilitySystemManager.shared.getUtility(PerformanceUtilityManager.self)
perfManager?.startProfiling(identifier: "text_processing")
// ... perform operations
let result = perfManager?.stopProfiling(identifier:
    "text_processing")
```

21.3.4 Concurrency

```
let concurrency =
    UtilitySystemManager.shared.getUtility(ConcurrencyUtilities.self)
let task = concurrency?.createTask {
    // Background work
    return processLargeFile()
}
let result = await task?.value
```

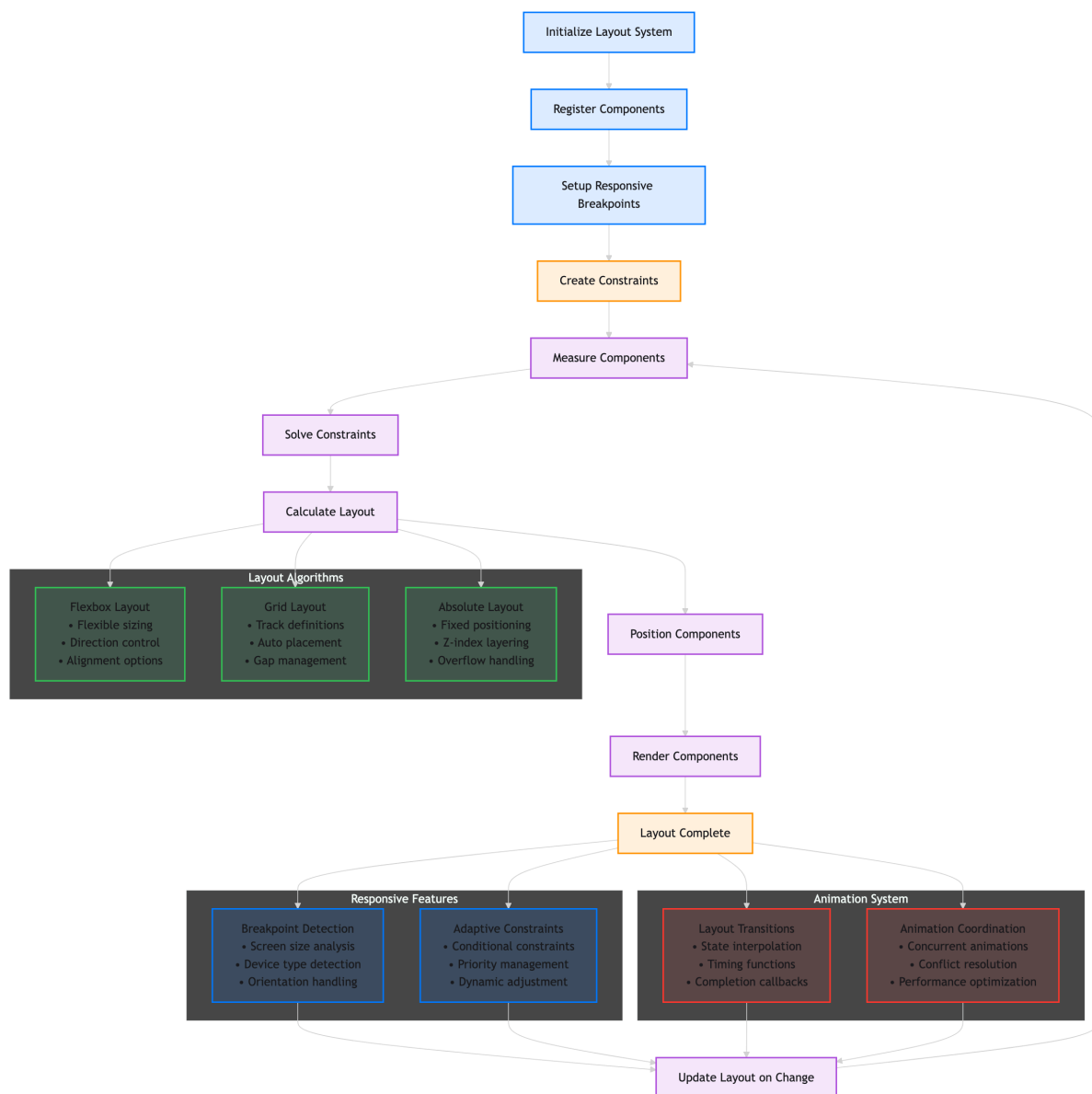
21.4 Benefits

1. **Unified Access:** Single point of access for all utility functions
2. **Cross-Platform:** Consistent behavior across macOS, iOS, and Catalyst
3. **Extensible:** Plugin architecture for custom utilities
4. **Performance:** Optimized implementations with caching and lazy loading
5. **Thread-Safe:** Comprehensive concurrency support throughout
6. **Maintainable:** Modular design with clear separation of concerns

22 Advanced Layout & UI Components Architecture

This diagram shows the comprehensive layout system and UI component architecture that handles advanced positioning, responsive design, and complex component interactions within the CodeEditorPlugin.

22.1 Layout System Flow



22.2 Key Layout System Features

22.2.1 1. Advanced Layout Algorithms

- **Flexbox Layout:** Flexible sizing with direction control and alignment options
- **Grid Layout:** CSS Grid-inspired layout with track definitions and auto placement
- **Absolute Layout:** Fixed positioning with z-index layering and overflow

handling

- **Flow Layout:** Traditional flow-based layout for text and inline elements

22.2.2 2. Responsive Design System

- **Breakpoint Management:** Screen size and device type responsive breakpoints
- **Adaptive Constraints:** Conditional constraints that adapt to different screen sizes
- **Orientation Handling:** Automatic layout adaptation for orientation changes
- **Device-Specific Optimizations:** Tailored layouts for different device capabilities

22.2.3 3. Advanced Constraint System

- **Linear Constraint Solver:** Efficient constraint solving with conflict detection
- **Priority-Based Resolution:** Constraint priority system for conflict resolution
- **Dynamic Constraints:** Runtime constraint modification and updates
- **Performance Optimization:** Cached constraint solutions and batch processing

22.2.4 4. Component Architecture

- **Protocol-Based Components:** Flexible component protocol for extensibility
- **Lifecycle Management:** Complete component lifecycle from creation to destruction
- **Factory Pattern:** Configurable component creation with dependency injection
- **Event System:** Comprehensive event handling and propagation

22.2.5 5. Animation and Transitions

- **Layout Animations:** Smooth transitions between layout states
- **Timing Functions:** Customizable animation timing and easing
- **Concurrent Animations:** Multiple simultaneous animations with coordination
- **Performance Optimization:** Hardware-accelerated animations where possible

22.2.6 6. Performance Optimizations

- **Measurement Caching:** Intelligent caching of component measurements
- **Dirty Region Tracking:** Minimal redraws using dirty region optimization
- **Batch Processing:** Batched layout calculations for improved performance
- **Memory Management:** Efficient memory usage and cleanup

22.2.7 7. Accessibility Integration

- **Accessibility Elements:** Automatic accessibility element generation
- **Focus Management:** Keyboard and assistive technology focus handling
- **Navigation Assistance:** Screen reader navigation support
- **Dynamic Updates:** Accessibility updates during layout changes

22.3 Editor-Specific Components

22.3.1 1. Code Editor Component

- **Text Rendering:** High-performance text rendering with syntax highlighting
- **Selection Management:** Text selection with multi-cursor support
- **Scrolling Integration:** Smooth scrolling with momentum and elastic behavior
- **Overlay System:** Flexible overlay system for annotations and UI elements

22.3.2 2. Gutter Component

- **Line Numbers:** Efficient line number rendering with customizable formatting
- **Breakpoint Visualization:** Interactive breakpoint display and management
- **Code Folding:** Visual code folding indicators with expand/collapse
- **Annotation Display:** Rich annotation display with hover interactions

22.3.3 3. Minimap Component

- **Content Overview:** Miniature view of entire document content
- **Viewport Indicator:** Visual indicator of current viewport position
- **Navigation Interface:** Click and drag navigation through document
- **Performance Optimization:** Efficient rendering with content caching

22.3.4 4. Scroll Component

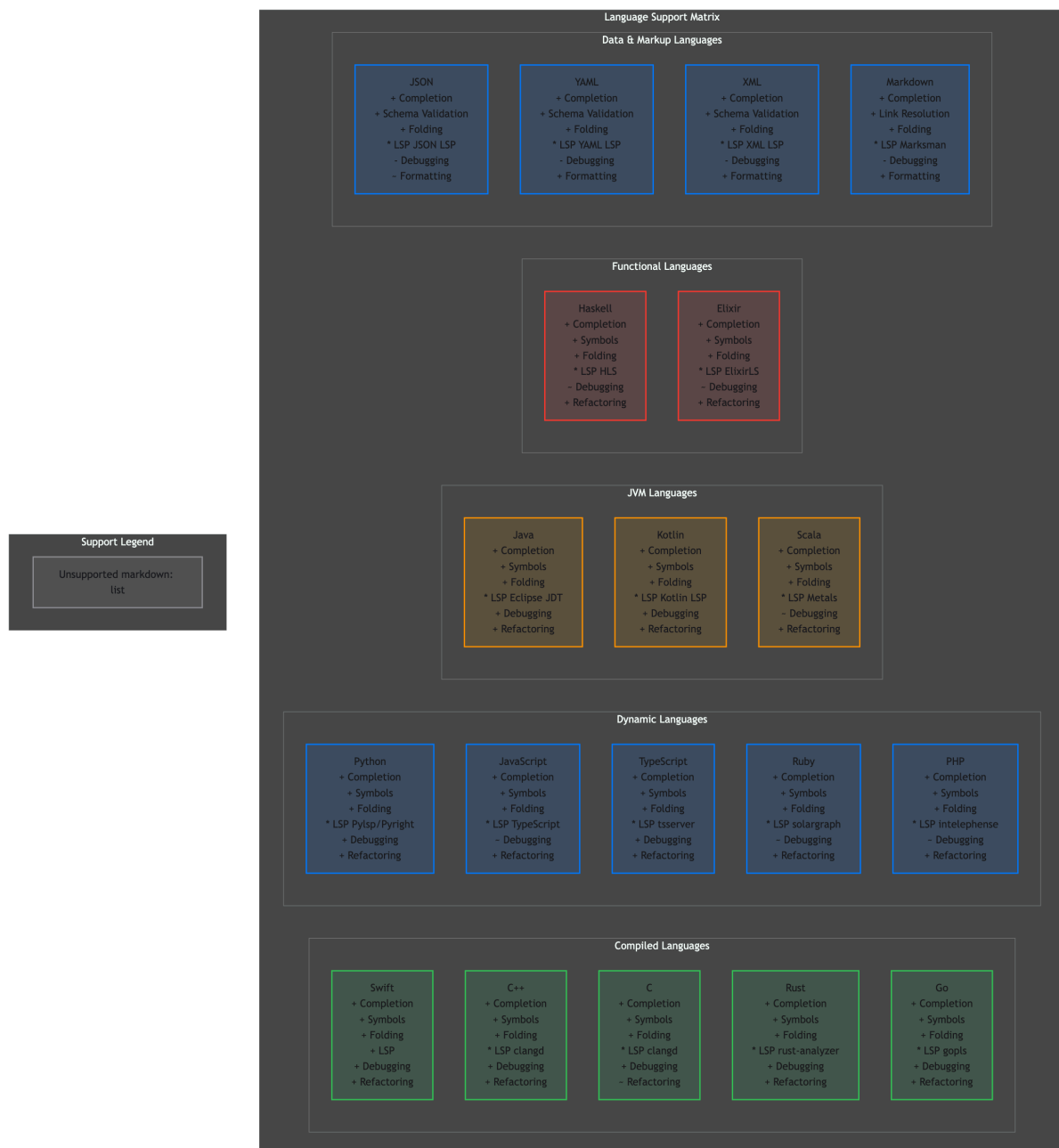
- **Smooth Scrolling:** Hardware-accelerated smooth scrolling
- **Zoom Support:** Pinch-to-zoom with content scaling
- **Elastic Behavior:** Natural scrolling behavior with bounce effects
- **Scroll Indicators:** Customizable scroll bar appearance and behavior

22.4 Benefits

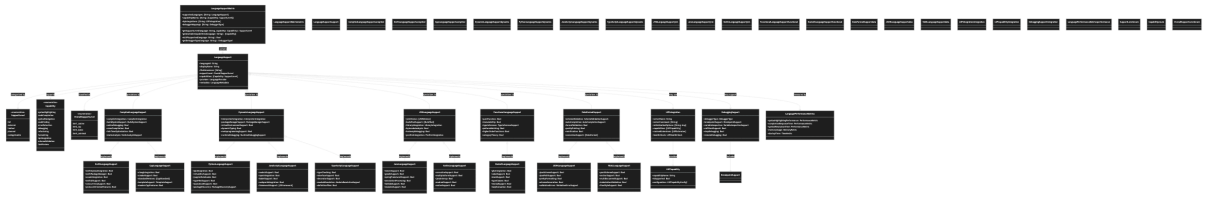
1. **Flexible Layout:** Support for multiple layout algorithms and responsive design
2. **High Performance:** Optimized layout calculations with caching and batching
3. **Smooth Animations:** Hardware-accelerated animations with conflict resolution
4. **Accessibility:** Built-in accessibility support with comprehensive navigation
5. **Extensible:** Protocol-based architecture allows for custom components
6. **Cross-Platform:** Consistent behavior across macOS, iOS, and Catalyst

23 Multi-Language Support Matrix

This diagram provides a comprehensive matrix view of language support capabilities across all 17+ supported languages in the CodeEditorPlugin framework.



23.1 Language Capability Details



23.2 Language Support Tiers

23.2.1 Tier 1: Native Support (Full Integration)

- **Swift:** Complete native integration with SwiftSyntax
- **C/C++:** Full Clang-based parsing and analysis
- **JSON/YAML:** Native parsing with schema validation

23.2.2 Tier 2: LSP Integration (External Server)

- **TypeScript/JavaScript:** Via TypeScript Language Server
- **Python:** Via Pylsp or Pyright
- **Rust:** Via rust-analyzer
- **Go:** Via gopls
- **Java:** Via Eclipse JDT Language Server
- **Kotlin:** Via Kotlin Language Server

23.2.3 Tier 3: Basic Support (Pattern Based)

- **Ruby:** Basic syntax highlighting and completion
- **PHP:** Pattern-based with limited completion
- **Scala:** Basic support with Metals LSP option

23.2.4 Tier 4: Minimal Support (Syntax Only)

- **Markdown:** Basic syntax highlighting with link resolution

- **XML:** Basic parsing with schema validation

23.3 Feature Comparison Matrix

Language	Completion	Symbols	Folding	LSP	Debugging	Refactorin
Swift	✓ Native	✓ Full	✓ Full	✓ Built-in	✓ LLDB	✓ Full
C++	✓ Clang	✓ Full	✓ Full	⚡ clangd	✓ GDB/ LLDB	✓ Full
C	✓ Basic	✓ Full	✓ Full	⚡ clangd	✓ GDB/ LLDB	⚠ Limited
Rust	✓ Full	✓ Full	✓ Full	⚡ rust- analyzer	✓ GDB/ LLDB	✓ Full
Go	✓ Full	✓ Full	✓ Full	⚡ gopls	✓ Delve	✓ Full
Python	✓ Full	✓ Full	✓ Full	⚡ Pylsp/ Pyright	✓ pdb	✓ Full
JavaScript	✓ Full	✓ Full	✓ Full	⚡ tsserver	⚠ Node.js	✓ Full
TypeScript	✓ Full	✓ Full	✓ Full	⚡ tsserver	✓ Node.js	✓ Full
Java	✓ Full	✓ Full	✓ Full	⚡ Eclipse JDT	✓ JDB	✓ Full
Kotlin	✓ Full	✓ Full	✓ Full	⚡ Kotlin LSP	✓ JDB	✓ Full
Scala	✓ Basic	✓ Limited	✓ Full	⚡ Metals	⚠ JDB	✓ Limited
Haskell	✓ Full	✓ Full	✓ Full	⚡ HLS	⚠ GHCi	✓ Full
Elixir	✓ Full	✓ Full	✓ Full	⚡ ElixirLS	⚠ IEx	✓ Full
Ruby	✓ Basic	✓ Limited	✓ Full	⚡ Solargraph	⚠ byebug	✓ Limited

Language	Completion	Symbols	Folding	LSP	Debugging	Refactorin
PHP	✓ Basic	✓ Limited	✓ Full	⚡ Intelephense	⚠ Xdebug	✓ Limited
JSON	✓ Schema	✓ Path	✓ Full	⚡ JSON LSP	✗ N/A	⚠ Format
YAML	✓ Schema	✓ Anchor	✓ Full	⚡ YAML LSP	✗ N/A	✓ Format
XML	✓ Schema	✓ Element	✓ Full	⚡ XML LSP	✗ N/A	✓ Format
Markdown	✓ Link	✓ Header	✓ Full	⚡ Marksman	✗ N/A	✓ Format

23.4 Performance Characteristics

23.4.1 High Performance Languages (Native Integration)

- **Swift:** Direct AST parsing, < 50ms completion response
- **C/C++:** Clang-based parsing, < 100ms completion response
- **JSON/YAML:** Native parsing, < 10ms validation

23.4.2 Medium Performance Languages (LSP Integration)

- **TypeScript:** External server, 100-300ms response time
- **Python:** External server, 150-400ms response time
- **Rust:** rust-analyzer, 200-500ms response time

23.4.3 Basic Performance Languages (Pattern Based)

- **Ruby/PHP:** Pattern matching, 50-150ms response time
- **Markdown:** Simple parsing, < 50ms response time

23.5 Extension Strategy

23.5.1 Adding New Languages

1. **Assess Support Level:** Determine appropriate tier
2. **Implementation Path:** Choose native, LSP, or pattern-based approach
3. **Capability Mapping:** Define supported capabilities
4. **Performance Targets:** Set performance expectations
5. **Testing Strategy:** Create comprehensive test suite

23.5.2 LSP Integration Process

1. **Server Discovery:** Identify available language servers
2. **Capability Negotiation:** Map LSP capabilities to framework features
3. **Configuration:** Set up initialization and workspace configuration
4. **Health Monitoring:** Implement server health checks
5. **Fallback Strategy:** Define behavior when LSP is unavailable

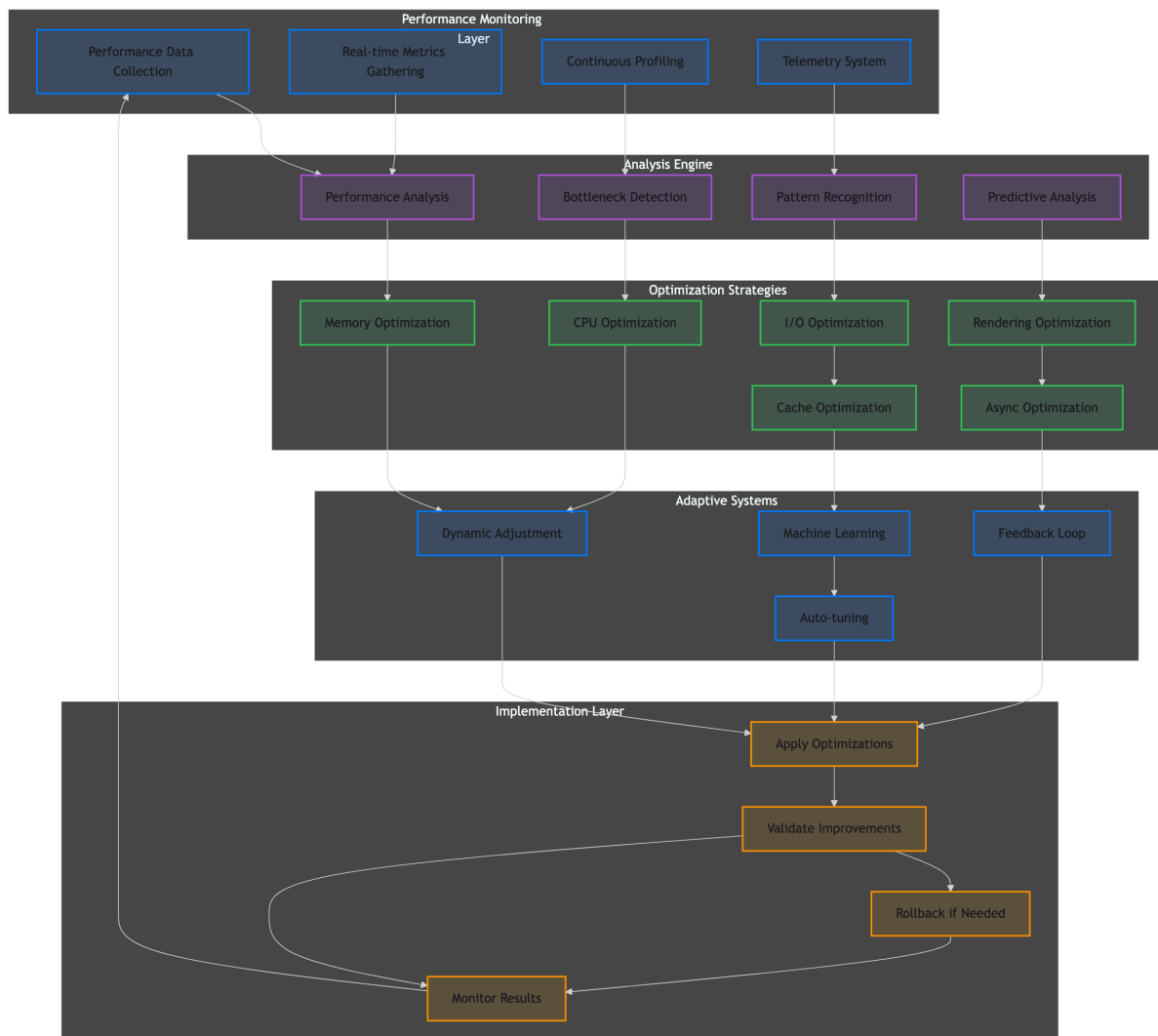
23.6 Benefits

1. **Comprehensive Coverage:** Support for 17+ major programming languages
2. **Flexible Architecture:** Multiple integration approaches for different needs
3. **Performance Optimized:** Appropriate performance targets for each language
4. **Extensible Design:** Easy addition of new languages and capabilities
5. **Standards Compliant:** LSP integration ensures compatibility with ecosystem tools
6. **Consistent Experience:** Unified API despite different underlying

implementations

24 Performance Optimization Pipeline

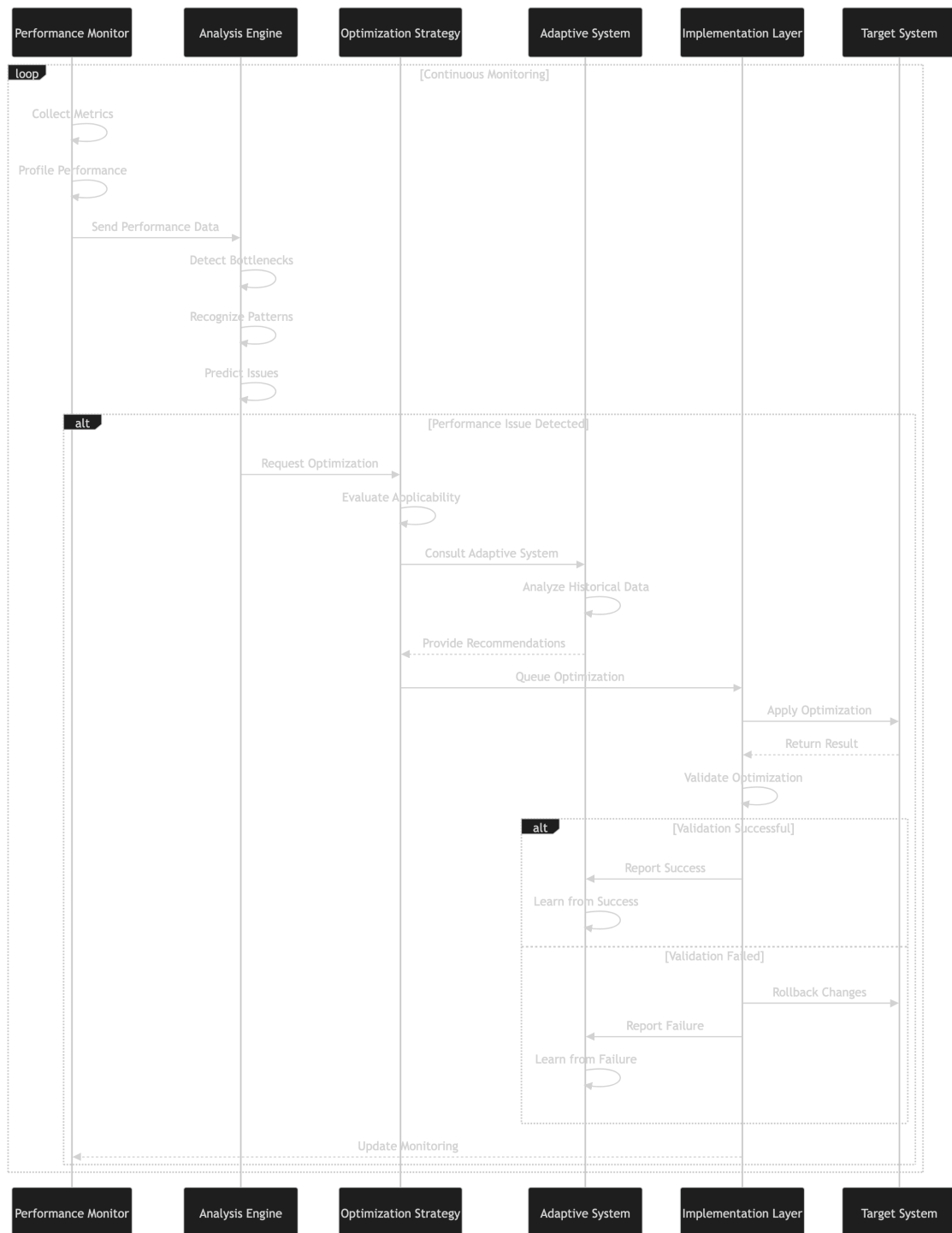
This diagram shows the comprehensive performance optimization pipeline that monitors, analyzes, and continuously optimizes performance across all aspects of the CodeEditorPlugin framework.



24.1 Detailed Performance Architecture



24.2 Performance Optimization Flow



24.3 Key Performance Features

24.3.1 1. Comprehensive Monitoring

- **Real-time Metrics:** Continuous collection of CPU, memory, I/O, and rendering metrics
- **Continuous Profiling:** Always-on profiling with minimal overhead
- **Telemetry System:** Privacy-respecting telemetry for performance insights
- **Alert System:** Proactive alerts for performance degradation

24.3.2 2. Advanced Analysis

- **Bottleneck Detection:** Multi-algorithm bottleneck identification
- **Pattern Recognition:** ML-powered pattern recognition for performance issues
- **Predictive Analysis:** Forecast performance problems before they occur
- **Trend Analysis:** Long-term performance trend identification

24.3.3 3. Multi-Strategy Optimization

- **Memory Optimization:** Advanced memory management and allocation strategies
- **CPU Optimization:** Algorithm optimization and concurrency improvements
- **Rendering Optimization:** Graphics pipeline and viewport optimizations
- **I/O Optimization:** File system and network performance improvements

24.3.4 4. Adaptive Learning

- **Dynamic Adjustment:** Real-time parameter adjustment based on conditions
- **Machine Learning:** Reinforcement learning for optimization strategies
- **Feedback Loop:** Continuous improvement through result analysis
- **Auto-tuning:** Automatic parameter tuning for optimal performance

24.3.5 5. Safe Implementation

- **Rollback Management:** Safe rollback of failed optimizations
- **Validation System:** Comprehensive validation of optimization results
- **Conflict Resolution:** Intelligent resolution of optimization conflicts
- **Snapshot System:** Performance state snapshots for safe rollback

24.3.6 6. Performance Targets

- **60fps Rendering:** Maintain 60fps during all operations
- **Sub-100ms Response:** UI responsiveness under 100ms
- **Memory Efficiency:** Optimal memory usage with minimal fragmentation
- **Scalability:** Performance scales with document size and complexity

24.4 Optimization Strategies

24.4.1 Memory Optimization

- **Object Pooling:** Reuse objects to reduce allocation overhead
- **Memory Mapping:** Use memory-mapped files for large documents
- **Cache Optimization:** Intelligent caching with LRU eviction
- **Garbage Collection:** Optimize GC timing and frequency

24.4.2 CPU Optimization

- **Algorithm Selection:** Choose optimal algorithms based on data size
- **Concurrency:** Leverage multi-core processing effectively
- **Vectorization:** Use SIMD instructions for parallel operations
- **Load Balancing:** Distribute work across available cores

24.4.3 Rendering Optimization

- **Viewport Culling:** Only render visible content

- **Batching:** Batch rendering operations to reduce overhead
- **Level of Detail:** Adjust rendering quality based on zoom level
- **Hardware Acceleration:** Leverage GPU for appropriate operations

24.4.4 I/O Optimization

- **Asynchronous Operations:** Non-blocking I/O operations
- **Compression:** Compress data to reduce I/O overhead
- **Prefetching:** Anticipate and preload required data
- **Caching:** Cache frequently accessed data

24.5 Benefits

1. **Continuous Improvement:** Performance automatically improves over time
2. **Proactive Optimization:** Problems are prevented before they impact users
3. **Adaptive Behavior:** System learns and adapts to usage patterns
4. **Safe Optimizations:** Comprehensive validation and rollback capabilities
5. **Comprehensive Coverage:** Optimizes all aspects of system performance
6. **Data-Driven Decisions:** Optimization decisions based on real performance data

25 Color Palette – Universal Light/Dark Mode

25.1 Core System Colors (Light/Dark Mode Compatible)

Descriptive Name	Light Mode Value	Dark Mode Value	Use Case	Category
Primary Text	#1D1D1F	#F2F2F7	Main text content	Text
Secondary Text	#3A3A3C	#EBEBF5	Subtitle, description text	Text
Tertiary Text	#48484A	#EBEBF5 (60% opacity)	Placeholder, disabled text	Text
Primary Background	#FFFFFF	#000000	Main background	Background
Secondary Background	#F2F2F7	#1C1C1E	Card, panel backgrounds	Background
Tertiary Background	#F2F2F7 (50% opacity)	#2C2C2E	Grouped content	Background
Separator	#3C3C43 (29% opacity)	#54545899	Dividers, borders	Border
Border Light	#3C3C43 (12% opacity)	#545458 (20% opacity)	Subtle borders	Border
Border Medium	#3C3C43 (36% opacity)	#545458 (65% opacity)	Prominent borders	Border

25.2 Accent Colors (Adaptive)

Descriptive Name	Light Mode Value	Dark Mode Value	Use Case	Category
Primary Blue	#007AFF	#0A84FF	Links, primary actions	Blue
Blue Background	#007AFF (10% opacity)	#0A84FF (20% opacity)	Blue backgrounds	Blue
Success Green	#34C759	#30D158	Success states	Green
Green Background	#34C759 (10% opacity)	#30D158 (20% opacity)	Success backgrounds	Green
Warning Orange	#FF9500	#FF9F0A	Warning states	Orange
Orange Background	#FF9500 (10% opacity)	#FF9F0A (20% opacity)	Warning backgrounds	Orange
Error Red	#FF3B30	#FF453A	Error states	Red
Red Background	#FF3B30 (10% opacity)	#FF453A (20% opacity)	Error backgrounds	Red
Purple Accent	#AF52DE	#BF5AF2	Purple accents	Purple
Purple Background	#AF52DE (10% opacity)	#BF5AF2 (20% opacity)	Purple backgrounds	Purple

25.3 Neutral Grays (Adaptive)

Descriptive Name	Light Mode Value	Dark Mode Value	Use Case	Category
Gray 1	#8E8E93	#8E8E93	Icons, secondary elements	Gray
Gray 2	#AEAEB2	#636366	Disabled state	Gray
Gray 3	#C7C7CC	#48484A	Subtle backgrounds	Gray
Gray 4	#D1D1D6	#3A3A3C	Input backgrounds	Gray
Gray 5	#E5E5EA	#2C2C2E	Panel backgrounds	Gray
Gray 6	#F2F2F7	#1C1C1E	System backgrounds	Gray

25.4 Semantic Colors (Context-Aware)

Descriptive Name	Light Mode Value	Dark Mode Value	Use Case	Category
Code Background	#F6F8FA	#161B22	Code blocks	Code
Code Text	#24292F	#F0F6FC	Code text	Code
Code Comment	#6E7781	#8B949E	Code comments	Code
Code Keyword	#CF222E	#FF7B72	Code keywords	Code
Code String	#0A3069	#A5D6FF	Code strings	Code
Code Function	#8250DF	#D2A8FF	Code functions	Code
Highlight Background	#FFF8C5	#FFEAA7 (30% opacity)	Search highlights	Highlight
Selection Background	#007AFF (20% opacity)	#0A84FF (30% opacity)	Text selection	Selection

25.4.1 How to use this palette

- **Light/Dark Mode:** Each color has specific light and dark mode values optimized for readability and accessibility
- **Semantic Usage:** Use colors based on their intended purpose (e.g., Error Red for error states)
- **Contrast:** All colors meet WCAG AA accessibility standards for their

intended use cases

- **Consistency:** Use the same semantic color across all diagrams for consistent meaning

Implementation Note: When creating diagrams, use the appropriate mode colors based on your target audience or provide both variants for maximum compatibility.

26 CodeEditorPlugin Architecture Diagrams

This directory contains **24 comprehensive architectural diagrams** for the CodeEditorPlugin framework. These diagrams provide complete coverage of all major components, systems, and integrations, illustrating their relationships and data flows throughout the entire system.

26.1 Index of Diagrams

26.1.1 1. High Level Architecture

Overview of the entire CodeEditorPlugin framework showing main layers and their relationships. Includes SwiftUI integration, core components, services, configuration, platform abstraction, features, language support, and external integrations.

26.1.2 2. Core Components Class Diagram

Detailed class diagram of the main components including CodeEditorView, CodeEditorAPI protocol, UnifiedEventSystem, BusinessLogicServiceRegistry, and their relationships. Shows the protocol-oriented design and delegation patterns.

26.1.3 3. Configuration System

Complete configuration architecture including EditorConfiguration structure with Display, Layout, Behavior, and Performance sections. Shows presets, validation, persistence, and SwiftUI environment integration.

26.1.4 4. Service Architecture

Service-oriented architecture diagram showing BusinessLogicServiceRegistry and all managed services (TextEditingService, SyntaxHighlightingService, LanguageDetectionService, CompletionManager, MemoryMonitor). Includes lifecycle management and event integration.

26.1.5 5. Event System Flow

Unified event system flow diagram illustrating event sources, creation, filtering, queuing, dispatching, and handler execution. Shows priority-based processing and async support.

26.1.6 6. Language Support & Syntax Highlighting Pipeline

Complete pipeline for language detection and syntax highlighting, including both SwiftSyntax and regex-based paths. Shows caching, tokenization, and rendering stages.

26.1.7 7. Completion System Architecture

Code completion system including CompletionManager, provider registry, session management, caching, and UI components. Includes sequence diagram of completion flow.

26.1.8 8. Platform Abstraction Layer

Cross-platform compatibility layer showing abstractions for macOS, iOS, and Mac Catalyst. Includes platform detection, type aliases, event/input adapters, and platform-specific implementations.

26.1.9 9. Text Processing Pipeline

Text processing flow from input to rendering, including TextKit2 integration, line index management, batch processing, and performance optimizations. Shows incremental updates and viewport rendering.

26.1.10 10. UI Component Hierarchy

Visual component hierarchy showing CodeEditorContainerView and all child components including gutter, minimap, overlays, and status bar. Includes layout structure diagram.

26.1.11 11. Advanced Features Integration Architecture

Comprehensive architecture for advanced features including debugging integration, search functionality, smart editing, and code folding. Shows feature coordination, state management, and UI integration.

26.1.12 12. LSP System Complete Architecture

Complete Language Server Protocol implementation with transport layers, protocol integration, and multi-language support. Includes message routing, capability negotiation, and error handling.

26.1.13 13. Performance Monitoring & Optimization System

Advanced performance monitoring with adaptive optimization, memory management, and real-time metrics. Includes profiling, bottleneck detection, and automatic performance tuning.

26.1.14 14. Symbol Navigation & Code Intelligence

Symbol navigation and code intelligence system with multi-language support, cross-reference tracking, and intelligent navigation. Includes definition lookup, reference finding, and workspace symbol search.

26.1.15 15. Language Provider Complete Ecosystem

Comprehensive language provider ecosystem supporting 17+ languages with completion, symbols, folding, and data providers. Shows the complete matrix of supported languages and their capabilities.

26.1.16 16. Annotation System Detailed Architecture

Comprehensive annotation system providing code annotations, diagnostics, and contextual information overlay capabilities. Includes multi-source annotation support, interactive features, and visual customization.

26.1.17 17. Advanced Text Processing & Validation Pipeline

Advanced text processing and validation system with multi-phase validation, range management, text versioning, and flexible styling. Includes performance optimization and comprehensive validation approaches.

26.1.18 18. Data Models & Type System Architecture

Comprehensive data models and type system forming the foundation of CodeEditorPlugin's data structures. Includes rich text models, versioning system, type information, and performance optimization.

26.1.19 19. SwiftUI Integration Complete Ecosystem

Complete SwiftUI integration ecosystem providing seamless integration between CodeEditorPlugin and SwiftUI applications. Includes platform-specific representables, environment management, and animation coordination.

26.1.20 20. Debugging Integration Detailed Architecture

Comprehensive debugging integration system providing breakpoint management, debug session control, and debugging visualization capabilities. Supports multiple debuggers including LLDB, GDB, and Debug Adapter Protocol.

26.1.21 21. Utility Systems & Extensions Network

Comprehensive utility systems and extensions network providing shared utilities, cross-platform helpers, and extensibility infrastructure. Includes file system helpers, cryptography, networking, performance utilities, and extension management.

26.1.22 22. Advanced Layout & UI Components Architecture

Advanced layout system and UI component architecture handling positioning, responsive design, and complex component interactions. Includes flexbox/grid layouts, constraint solving, animation coordination, and accessibility integration.

26.1.23 23. Multi-Language Support Matrix

Comprehensive matrix view of language support capabilities across all 17+ supported languages. Shows feature comparison, performance characteristics, LSP integrations, and debugging support for each language.

26.1.24 24. Performance Optimization Pipeline

Comprehensive performance optimization pipeline that monitors, analyzes, and continuously optimizes performance. Includes real-time monitoring, bottleneck detection, adaptive optimization strategies, and machine learning-based improvements.

26.2 How to View These Diagrams

All diagrams are written in Mermaid syntax and can be viewed: 1. Directly in GitHub (automatic rendering) 2. In VS Code with a Mermaid preview extension 3. In any Mermaid-compatible viewer 4. Exported to SVG/PNG using Mermaid CLI tools

26.3 Diagram Conventions

All diagrams use a consistent light/dark mode compatible color palette:

- **Primary Blue:** Core components/main systems
- **Success Green:** UI/visual components
- **Purple Accent:** Services/processing components
- **Warning Orange:** Configuration/settings
- **Error Red:** External integrations
- **Neutral Gray:** Supporting data types and enums
- **Arrows:** Dependencies and data flow
- **Subgraphs:** Logical groupings of related components

Colors automatically adapt to light/dark mode with semantic meaning maintained across all diagrams.